Decision-Oriented Enforcement of Parametric Content-Based Row Level Policies

Nilanjan Ghosh
Indian Institute of Technology Delhi
nilanjan.ghosh@cse.iitd.ac.in

Kaustubh Beedkar
Indian Institute of Technology Delhi
kbeedkar@cse.iitd.ac.in

Abstract

Access control is a fundamental component of database systems. In this paper, we study parametric content-based row-level policies, in which the visibility of a tuple depends on the existence or absence of related tuples in other relations, and whose comparison operators and constants are bound only at query time. Existing mechanisms instantiate such a policy per query and compile it into a conventional plan of joins, semijoins, and antijoins, which often leads to wasted work. We propose a decision-oriented policy evaluation, which makes decisions rather than tuples the unit of computation. Our approach compiles an instantiated policy into an AND/OR decision graph and decides each target tuple by a depth-first walk that stops at the first witness. Along the way, it memoizes each decision under a compact key of the values it depends on, so that later tuples reuse it instead of re-reading the data. To perform this efficiently, we execute over dense, order-preserving ranks, which let a single set of indexes serve every policy instantiation and let one decision answer all keys on the same side of a threshold. Our experiments on real and synthetic datasets demonstrate that our approach can achieve more than an order of magnitude improvement over traditional approaches.

PVLDB Reference Format:
Nilanjan Ghosh and Kaustubh Beedkar. Decision-Oriented Enforcement of Parametric Content-Based Row Level Policies. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/IITD-data-systems/parametric-cbac.

1 Introduction

Modern systems increasingly enforce access control at the level of individual rows, in accordance with security, privacy, and regulatory requirements [8]. In multi-tenant services, clinical systems, and data-sharing platforms, the visibility of a tuple may depend on the requester, the context of the request, and the contents of the data itself [3, 31]. Every major engine therefore provides fine-grained enforcement, including row-level security in PostgreSQL [54] and SQL Server [34], row access policies in Snowflake [50] and BigQuery [20], and Oracle VPD [43].

Content-based row level policies are particularly demanding to enforce. Under such policies, the visibility of a tuple in a target relation is determined by the existence, or absence, of related tuples in other relations [45, 48]. Consider an order-management database (Figure 1) comprising `orders`, `customer`, `lineitem`, and `supplier` relations. A procurement policy may expose an order only if (i) at least one of its lineitems is sourced from a supplier located in the nation of the order's customer, and (ii) no lineitem of the order could be fulfilled by a supplier that failed its compliance audit. Condition (i) is *existential*: it requires a witness that spans three relations and closes a cycle through the nation attribute. Condition (ii) is *anti-existential*: it requires certified absence of any witness over `lineitem` and `supplier`. Neither condition can be decided from the order tuple alone. Such combinations of grants and overriding prohibitions are common in access control policies [4, 13, 39].

Such policies are rarely fixed at definition time. An organization typically applies one policy *shape* to many roles, tenants, and requests that differ only in how strict its conditions are: the audit status to exclude, a request-specific threshold, or the comparison that relates a supplier's capacity to a lineitem's quantity, e.g., $\geq$ to ask whether a supplier *could* fulfill a lineitem and $=$ to ask whether its capacity matches the quantity exactly. Instead of maintaining one policy per variant, the application or the session context supplies these choices when the query is issued [22, 26, 42]. We refer to this setting as *parametric content-based row-level access control*: a policy is a template whose comparison operators and constants are bound *only* at query time.

Parametricity makes enforcement challenging for two reasons. First, because the policy predicate is not fully determined until query time, authorization results cannot be precomputed and must be derived for each request. Second, a rebinding alters the selectivity and the structure of the predicate. Rebinding $\texttt{s_cap} \geq \texttt{l_qty}$ to $\texttt{s_cap} = \texttt{l_qty}$, for instance, turns a theta join into an equi-join, so join order, filter placement, and physical operator choice may depend on the binding. A plan optimized for one instantiation can therefore be arbitrarily poor for the next, and errors in cardinality estimation compound the problem [32]. Caching a plan per binding is also impractical as the number of possible bindings grows exponentially with the number of operator variables, each potentially requiring a different plan.

Existing solutions span production row-level security [20, 34, 50, 54], authorization views [48], table functions [55], query rewriting [52], and access-control middleware such as Sieve [45], Data Guard [56], and Hasura [21]. Declarative policy languages such as Cedar [13] and the Open Policy Agent [41] decide requests over entity data supplied by the application, and are translated into database filters when a policy depends on database content. Despite differences in interface and deployment, these systems share a single enforcement strategy. They instantiate the template at query time, compile the resulting predicate into a conventional execution

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

plan, and delegate evaluation to the engine's join and filter machinery. In effect, *authorization is reduced to query evaluation.* A policy that references several relations becomes a multi-way pipeline of joins, semijoins, and antijoins, which is optimized and executed anew for every request.

This reduction rests on a fundamental mismatch. Authorization under a content-based policy is a *decision problem*: for each target tuple, the engine must determine whether the required witnesses exist and the forbidden ones do not. A single witness suffices to establish existence or disprove absence, and the witnesses themselves are never part of the answer. Tuple-producing plans, however, evaluate these conditions by producing qualifying witness combinations and often materializing intermediate results whose only role is to establish the decision.

We identify three sources of wasted work in tuple-producing enforcement. **(1) Witness blowup.** Joins retain all matching combinations although one witness suffices. First-match index nested-loop semijoins avoid this, but are rarely chosen for multi-relation policies and do not address (2) or (3). **(2) Repeated witness search.** Correlated plans repeat searches for each target tuple. Caching reuses results only for identical correlation keys, missing shared outcomes across distinct keys, particularly under ordered comparisons. Decorrelation [37] instead reintroduces the joins of (1). **(3) Duplicated reads across polarities.** Since $\exists A \wedge \neg \exists B$ is not equivalent to $\exists(A \wedge \neg B)$, existential and anti-existential conditions become separate semijoin and antijoin branches that independently access shared relations.

We propose decision-oriented policy evaluation, which makes *decisions*, rather than tuples, the unit of computation and reuses them across target tuples. We compile an instantiated policy into a *decision graph* that combines Boolean conditions with checks for the existence or absence of witnesses. For each target tuple, a depth-first walk follows the policy's comparisons through the data and stops at the first witness. The walk memoizes each *witness outcome*, recording whether a witness exists under a *key* formed from the attribute values on which the decision depends. Later walks reaching the same key reuse the outcome without repeating the search, regardless of whether the policy requires existence or absence. This adapts context-based caching of AND/OR search [15] to relational witnesses. Whereas prior work reuses the tuples that joins produce, we reuse decisions to *avoid* the physical data access that would have produced them.

To make evaluation efficient across policy bindings, we introduce three database-specific techniques. First, we execute the walk over *ranks*. We map each value domain compared by the policy templates to a dense, order-preserving rank space, and we build forward and inverted indexes over these ranks once, offline. A bound operator merely selects a point, a range, or the complement of a point in rank space, so one set of artifacts and one evaluation strategy serve every binding without re-planning, and keys become compact integers. Second, on ordered comparisons, ranks enable *monotone outcomes*: an outcome learned for one key also holds for every key on the same side of it, so two boundary ranks decide keys that were never searched. Third, a *shared walk* reads the relations common to an existential and an anti-existential branch once, while each branch keeps its own outcome and polarity.

`customer`

rid	c_ck	c_nat
c_1	4	Fr
c_2	9	Jp

`orders`

rid	o_ok	o_ck
o_1	101	4
o_2	102	4
o_3	103	9

`lineitem`

rid	l_ok	l_sk	l_qty
ℓ_1	101	20	65
ℓ_2	101	21	90
ℓ_3	102	20	70
ℓ_4	102	22	60
ℓ_5	103	21	18
ℓ_6	103	22	50

`supplier`

rid	s_sk	s_nat	s_au	s_cap
s_1	20	Fr	P	30
s_2	21	Ge	P	30
s_3	22	Ge	F	60
s_4	23	Fr	F	40
s_5	24	Fr	P	90
s_6	25	Jp	P	30

Figure 1: Running database instance with four relations and abbreviated attributes. Each order belongs to a customer and contains lineitems specifying a supplier and quantity (`qty`). Customers and suppliers have a nation (`nat`); suppliers additionally have an audit status (`au`) (`P`: passed, `F`: failed) and a capacity (`cap`), the largest quantity they can fulfill.

We implement our approach, Memento, as a PostgreSQL extension. It enforces policies through an allowed-row bitmap at protected scans, without rewriting queries or modifying base tables. Since database updates require rebuilding its rank artifacts, the current implementation targets read-mostly analytical workloads. We evaluate Memento on 90 workloads based on TPC-H [57], JOB [32], and MIMIC [29]. On complex policies, Memento achieves a median 58× speedup over native row-level security and outperforms a hand-tuned DuckDB baseline on 21 of 30 workloads. Its artifacts occupy space comparable to conventional B-tree indexes.

Summary of contributions. We formalize parametric content-based row-level policies with mixed existential and anti-existential branches (Section 2) and characterize three sources of wasted work inherent to tuple-producing enforcement (Section 3). We propose *decision-oriented policy evaluation*, a demand-driven walk over a decision graph with key-based, polarity-independent memoization of witness outcomes (Section 4). We introduce *rank-based execution*, comprising binding-independent rank artifacts, monotone outcomes that generalize beyond observed keys, and shared walks across branches of opposite polarity (Section 5). Finally, we implement Memento as a PostgreSQL extension and evaluate it against production and hand-tuned baselines on synthetic and real-world workloads (Section 6). Section 7 discusses related work.

2 Preliminaries

Data Model. We consider a relational database instance $\mathcal{D}=\{R_1,..,R_n\}$. For each relation $R \in \mathcal{D}$, $\mathrm{sch}(R)$ denotes its finite set of attributes. Each tuple of R carries a surrogate row identifier, independent of its values, and we write $\mathrm{rids}(R)$ for the set of row identifiers of R and $R[r][A]$ for the value of $A \in \mathrm{sch}(R)$ of row $r \in \mathrm{rids}(R)$. Each attribute A takes values in a totally ordered domain $\mathrm{dom}(A)$ or the distinguished value NULL, which is incomparable to every value. Figure 1 shows an example database instance.

Parametric Content-Based Policies. A content-based policy governs each tuple's visibility in a *target* relation T. The tuple under consideration is the *anchor*; tuples of other relations justifying or preventing visibility are *witnesses*. Policies are Boolean combinations of witness existence (`EXISTS`) and absence (`NOT EXISTS`) conditions. A *parametric* policy additionally leaves its comparison operators and constants open until query time.[1] Let $\Omega=\{=,\neq,<,\leq,>,\geq\}$.

[1]We also allow the connective between branches ($\wedge/\vee$) and each branch's polarity ($\exists/\neg\exists$) to remain open until query time; we omit this for readability.

Operator variables $\omega_1,\omega_2,\ldots$ range over Ω, and *parameter variables* $\$1,\$2,\ldots$ range over attribute values; an unsubscripted $\omega \in \Omega$ always denotes a concrete operator. Each *tuple variable* x ranges over a fixed relation $\mathrm{rel}(x)$, with x_T denoting the anchor variable.

Definition 1 (Policy template). *A policy template π over a target relation T is generated by $\tau ::= x.A \mid \$j$ and $\alpha ::= \tau_1\, \omega_i\, \tau_2$, where*

$$\pi ::= \alpha \mid \pi_1 \wedge \pi_2 \mid \pi_1 \vee \pi_2 \mid \exists x_1,\ldots,x_k.\pi \mid \neg\exists x_1,\ldots,x_k.\pi.$$

Here, $x.A$ references attribute $A \in \mathrm{sch}(\mathrm{rel}(x))$, ω_i is an operator variable, and $\$j$ is a parameter variable. Each quantifier binds fresh tuple variables; x_T is never quantified and is the only free tuple variable of π. We write $\mathrm{opvars}(\pi)$ *and* $\mathrm{params}(\pi)$ *for the operator and parameter variables occurring in π.*

An atom thus compares either two attributes or an attribute with a parameter. Since every quantifier binds fresh variables, two occurrences of the same relation under different quantifiers denote independent witnesses.

Example 1 (Policy template). *For the running example, let o denote the target tuple variable x_T ranging over* `orders`*. The procurement policy of Section 1 is the template*

$$\pi = \big(\exists c,\ell,s.\big(\ell.l_ok\,\omega_1\,o.o_ok \wedge \ell.l_sk\,\omega_2\,s.s_sk \wedge o.o_ck\,\omega_3\,c.c_ck \wedge c.c_nat\,\omega_4\,s.s_nat\big)\big) \wedge \big(\neg\exists \ell',s'.\big(\ell'.l_ok\,\omega_5\,o.o_ok \wedge s'.s_cap\,\omega_6\,\ell'.l_qty \wedge s'.s_au\,\omega_7\,\$1\big)\big),$$

with $\mathrm{rel}(c)$ = `customer`, $\mathrm{rel}(\ell) = \mathrm{rel}(\ell')$ = `lineitem`, *and* $\mathrm{rel}(s) = \mathrm{rel}(s')$ = `supplier`*. It contains seven operator variables and one parameter variable. The lineitem ℓ witnessing the existential branch is unrelated to the lineitem ℓ' quantified in the anti-existential branch.*

Instantiation. At query time, a template is instantiated by a binding $\theta = (\theta_{\mathrm{op}}, \theta_{\mathrm{par}})$, where $\theta_{\mathrm{op}} : \mathrm{opvars}(\pi) \to \Omega$ and θ_{par} maps each parameter variable to a value compatible with the attributes it is compared against. The *instantiated policy* $\pi[\theta]$ is obtained from π by replacing every ω_i with $\theta_{\mathrm{op}}(\omega_i)$ and every $\$j$ with $\theta_{\mathrm{par}}(\$j)$. Its atoms thus have the form $\tau_1\,\omega\,\tau_2$ with $\omega \in \Omega$, where each term is an attribute reference $x.A$ or a value v. The binding is supplied per request, so a single template denotes a family of policies that may differ in selectivity and in the shape of their join conditions.

Semantics. An *assignment* b maps tuple variables to row identifiers, where $b(x) \in \mathrm{Wit}(\mathrm{rel}(x))$ and $\mathrm{Wit}(R) = \mathrm{rids}(R)$ is the *witness domain* of R (see Scope below for protected witness relations). We write $b[x \mapsto w]$ for the extension of b that maps x to w. Terms of an instantiated policy denote values, i.e., $[\![x.A]\!]_b = \mathrm{rel}(x)[b(x)][A]$ and $[\![v]\!]_b = v$. A formula denotes a truth value $[\![\pi]\!]_b \in \{0,1\}$:

$$[\![\tau_1\,\omega\,\tau_2]\!]_b = 1 \iff [\![\tau_1]\!]_b, [\![\tau_2]\!]_b \neq \mathrm{NULL} \text{ and } [\![\tau_1]\!]_b\,\omega\,[\![\tau_2]\!]_b,$$

$$[\![\exists x_1,\ldots,x_k.\pi']\!]_b = 1 \iff \exists w_1,\ldots,w_k : [\![\pi']\!]_{b[x_1\mapsto w_1,\ldots,x_k\mapsto w_k]} = 1,$$

$$[\![\neg\exists x_1,\ldots,x_k.\pi']\!]_b = 1 - [\![\exists x_1,\ldots,x_k.\pi']\!]_b,$$

where each w_i ranges over $\mathrm{Wit}(\mathrm{rel}(x_i))$, and $\wedge$ and $\vee$ are interpreted as usual. Because negation occurs only in front of quantifiers, this two-valued reading coincides with SQL, whose `WHERE` clause discards unknown comparisons and whose `EXISTS` is two-valued. The anchor $T[r]$ is *accessible* under $\pi[\theta]$ if $[\![\pi[\theta]]\!]_{\{x_T \mapsto r\}} = 1$, and the *accessible set* is

$$\mathrm{acc}(T,\pi,\theta) = \{r \in \mathrm{rids}(T) \mid [\![\pi[\theta]]\!]_{\{x_T\mapsto r\}} = 1\} \tag{1}$$

Evaluation starts from the assignment that binds only the anchor, and each quantifier extends it with its own witnesses.

Example 2 (Instantiation and accessibility). *Let θ bind $\omega_6 \mapsto \geq$, $\$1 \mapsto$ `F`, and all other operator variables to $=$. Suppliers s_3 (s_cap=60) and s_4 (s_cap=40) failed their audit. Then* $\mathrm{acc}(\texttt{orders},\pi,\theta) = \{o_1\}$*. Order o_1 is accepted: ℓ_1 is supplied by s_1, located in Fr like customer c_1, and its lineitems request 65 and 90, exceeding both failed suppliers' capacities. Order o_2 satisfies the existential branch via ℓ_3 and s_1, but is rejected because s_3 can fulfill ℓ_4's quantity of 60. Order o_3 fails the existential branch alone (customer in Jp, both suppliers in Ge), so its anti-existential branch need not be evaluated, which Section 4 exploits. The binding alters the decision: under $\omega_6 \mapsto =$, the result is unchanged since s_3 matches ℓ_4 exactly; under $\omega_6 \mapsto \leq$, s_3 witnesses ℓ_1 ($60 \leq 65$), and no order is accessible.*

Scope of the Policy Model. Our model supports arbitrary Boolean combinations and nesting of existential and anti-existential blocks containing comparison atoms and nested blocks. Atoms compare attributes with other attributes or parameters using operators in Ω; all operators and constants may be parametric. We exclude aggregation within quantifiers (e.g., `GROUP BY`, `HAVING`, "at least k lineitems"), arithmetic or functions within atoms, pattern predicates such as `LIKE`, outer joins, set operations, and recursion. If a witness relation R is itself protected, $\mathrm{Wit}(R)$ recursively denotes its allowed rows; policy dependencies must be acyclic. We present $\mathrm{Wit}(R) = \mathrm{rids}(R)$; the results extend by restricting candidate retrieval to allowed rows.

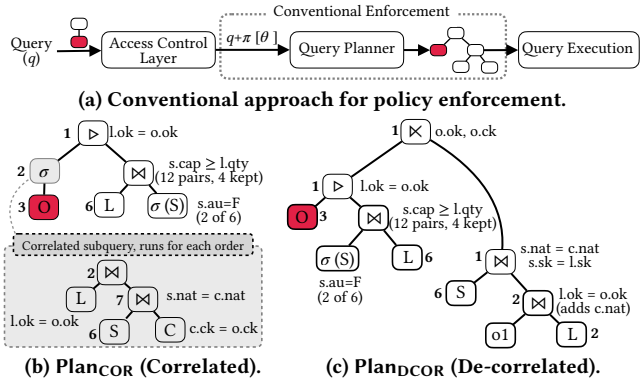
3 Conventional Policy Enforcement and Its Cost

Policy enforcement asks for one Boolean decision per target row, yet existing systems obtain it through conventional relational execution. We first describe this enforcement strategy and then use our running example to isolate three sources of wasted work. These motivate the requirements for our decision-oriented approach.

Enforcement in Current Systems. Figure 2a illustrates the conventional approach, in which systems attach policy predicates to scans of protected relations. For example, in PostgreSQL [54] the running policy can be expressed using row-level security, with audit status supplied through a session variable:

```
CREATE POLICY order_access ON orders USING (
  EXISTS (SELECT 1 FROM customer c, lineitem l, supplier s
          WHERE l.l_ok = orders.o_ok AND l.l_sk = s.s_sk
            AND orders.o_ck = c.c_ck AND c.c_nat = s.s_nat)
  AND NOT EXISTS (SELECT 1 FROM lineitem l2, supplier s2
          WHERE l2.l_ok = orders.o_ok AND s2.s_cap >= l2.l_qty
            AND s2.s_au = current_setting('app.audit')));
```

The engine combines this predicate with the user query, evaluating existential and anti-existential conditions through semijoins ($\ltimes$), antijoins ($\triangleright$), or correlated subplans. Existing mechanisms (e.g., [43, 45, 48, 56]) differ in how predicates reach the engine, but rely on the same evaluation machinery. Parametric operators have no counterpart in SQL; prepared-statement parameters cannot stand for operators. A system can emulate them in one of two ways. It can generate the policy predicate per request, for example by redefining the policy or by rewriting the query in middleware, which forces the engine to re-optimize for every binding. Alternatively, it can encode every admissible operator as a disjunction guarded by session variables, such as `(current_setting('app.op6') = '>=' AND s2.s_cap >= l2.l_qty) OR ...`, which hides the effective predicate from the optimizer and in general precludes index-based access paths. In either case, evaluation remains tuple-producing.



(a) Conventional approach for policy enforcement.

(b) Plan_{COR} (Correlated).

(c) $\text{Plan}_{\text{DCOR}}$ (De-correlated).

Figure 2: Overview of conventional policy enforcement (a). Two plans (b) & (c) for the instantiated policy of Example 2: each operator is annotated with its predicate and output cardinality, and each leaf with the number of tuples read.

Plans under Study. Given a query with an attached policy predicate, the planner may evaluate its subqueries using correlated execution or decorrelate them into joins and antijoins. Figures 2b and 2c illustrate these alternatives for Example 2, denoted Plan_{COR} and $\text{Plan}_{\text{DCOR}}$, respectively. We abbreviate relations as O, C, L, S and attributes (e.g., $o.ok$ for $o.o_ok$); primed variables denote independent witnesses in the negative branch. Plan_{COR} evaluates the existential branch per order and the negative branch as an antijoin over a band join of lineitems and failed suppliers. $\text{Plan}_{\text{DCOR}}$ decorrelates both branches, evaluating the antijoin first. We count accessed base tuples as *reads* and join outputs as *built tuples*, distinguishing predicate-tested pairs from join outputs.

(1) Witness blowup. An existential branch needs only one witness, while joins may produce many. The running branch is a diamond: the paths through C and L meet at $c.nat = s.nat$. One witness, (c_1, ℓ_1, s_1), authorizes o_1. Plan_{COR} instead first joins C and S by nation, pairing c_1 even with s_4, s_5, which supply no lineitem of o_1; its three subplan executions build seven such pairs, of which only two survive the later supplier–lineitem join. $\text{Plan}_{\text{DCOR}}$ joins the surviving order with L before S, retaining ℓ_2 even though ℓ_1 suffices. Joining O with C and L with S separately still produces three and six tuples. In the negative branch, the band join tests 12 lineitem–failed-supplier pairs, four of which qualify. At TPC-H scale factor 1, joining by nation can produce roughly 400 customer–supplier pairs per correlated execution to decide an order with only a few lineitems. First-match index nested-loop semijoins avoid this blowup when estimates favor them, but leave the next two costs; semijoin reduction, predicate transfer, or factorization can reduce or share intermediates without eliminating these costs together.

(2) Repeated witness search. Correlated plans re-search for each anchor; caching, such as PostgreSQL's Memoize, helps only when an identical correlation key recurs, while decorrelation restores the joins of (1). Plan_{COR} rebuilds the same three customer–supplier pairs for o_1 and o_2, which share c_1; its existential subplan reads 34 tuples in total from relations containing 14 tuples. More importantly, distinct keys can have related answers. The audit branch asks, for each of six distinct lineitem quantities, whether a failed supplier has sufficient capacity: exact-key caching cannot hit. Yet the maximum failed-supplier capacity is 60, so the outcomes at quantities 60 and 65 determine all six by monotonicity. Neither plan exploits this threshold. Even first-match techniques for diamond joins perform independent probes and carry no such knowledge between anchors.

(3) Duplicated reads across polarities. Both branches read lineitems and suppliers, but $\exists A \wedge \neg\exists B \neq \exists(A \wedge \neg B)$: the $\exists$ and $\neg\exists$ witnesses need not coincide. For o_2, ℓ_3 satisfies A and has no forbidden supplier, yet ℓ_4 has the forbidden witness s_3, so the order must be rejected. Independent semijoin/antijoin branches therefore repeat accesses to L and S (12 of $\text{Plan}_{\text{DCOR}}$'s 29 reads are second reads of L and S) . A shared mark-join scan with per-anchor flags and aggregation is possible, but still constructs join combinations. The duplication grows with the number of branches sharing relations.

Parametric rebinding and requirements. Rebinding adds another cost: the optimal join order can change with the bound operator. Regenerating a specialized predicate requires re-optimization; guarded disjunctions conceal the operator and can lose index access. Thus we need an approach that should (R1) stop at the first witness without constructing witness combinations; (R2) reuse outcomes for every key they determine, including keys on the same side of an ordered threshold; (R3) share reads across positive and negative branches without conflating their outcomes; and (R4) retain binding-independent evaluation and physical structures.

4 Decision-Oriented Policy Evaluation

We now discuss our decision-oriented approach. We first give an overview of Memento before describing the decision graph and the depth-first walk formally.

4.1 Overview

Figure 3 shows policy enforcement using Memento. Given a template π and a binding θ, Memento constructs a decision graph for $\pi[\theta]$, whose nodes represent Boolean subproblems rather than tuple-producing operators. For each anchor of the protected relation T, a depth-first walk binds witnesses one relation at a time, follows the comparisons of the policy, and stops at the first witness of each quantified subproblem, without constructing witness combinations (addressing R1). Each such subproblem records whether a witness exists under the values on which its outcome depends, and later arrivals, from the same anchor or a later one, reuse this decision instead of repeating the search (addressing R2). The decisions form an allowed-row bitmap that the protected scan applies, leaving the user query unchanged. Section 5 extends this basic walk with binding-independent rank artifacts, reuse beyond identical keys, and shared reads across branches (addressing R2–R4).

4.2 Decision Graph

We first put $\pi[\theta]$ into a normal form by standard miniscoping: each quantifier binds one tuple variable, and each atom lies in the scope of the innermost quantifier whose variable it references. Let $Qx.\varphi$ with $Q \in \{\exists, \neg\exists\}$ be a quantifier in this form. An atom is a *mandatory conjunct* of φ if it occurs in φ neither under a disjunction nor under a nested quantifier, so that every witness for x satisfies it. A mandatory conjunct that also references a variable bound outside the quantifier links x to the tuples bound so far; we designate one such conjunct as the *connection* of the quantifier, through which

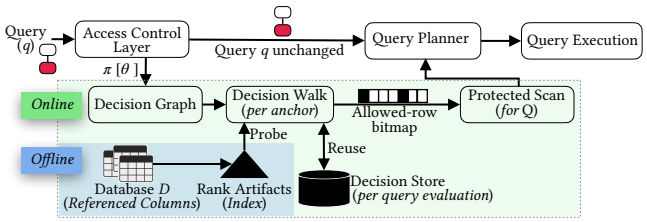


Figure 3: Overview of policy enforcement using Memento.

the walk retrieves candidates for x. A quantifier without one ranges over its entire witness domain.

Definition 2 (Decision graph). *The decision graph G of a normal form of $\pi[\theta]$ is the directed acyclic graph of its subformulas, with an edge from each subformula to its direct subformulas, in which subformulas equal up to a renaming of quantified variables over the same relations may share a node. Inner nodes are labeled $\wedge$, $\vee$, $\exists x$, or $\neg\exists x$, and leaves are atoms.*

A node n denotes a truth value $[\![n]\!]_b$ under an assignment b of its free tuple variables, and the decision for an anchor r is $[\![\mathrm{root}(G)]\!]_{\{x_T \mapsto r\}}$. Figure 4 shows the graph of the running policy. From an order o, the existential branch enters ℓ through $\ell.ok = o.ok$, s through $\ell.sk = s.sk$, and c through $c.nat = s.nat$; the mandatory conjunct $c.ck = o.ck$ is tested on each customer. Any designation of connections is correct (Theorem 1); entering c by nation follows the diamond of Section 3 (witness blowup). The audit branch enters ℓ' through $\ell'.ok = o.ok$ and s' through the band condition $s'.cap \geq \ell'.qty$, and tests $s'.au = \mathrm{F}$, which references only s', on each candidate. Although both branches enter `lineitem` by orderkey, each binds its own lineitem and keeps its own outcome.

4.3 The Walk and Its Decision Store

Candidates and keys. For $Qx.\varphi$, the walk examines candidate tuples of $\mathrm{rel}(x)$ and recursively tests φ on each until one succeeds or the candidates are exhausted.

Definition 3 (Candidates). *For a quantifier $Qx.\varphi$ and an assignment b, a candidate set $\mathrm{Cand}(x,\varphi,b)$ is any finite subset of $\mathrm{Wit}(\mathrm{rel}(x))$ that contains every tuple w such that $b[x \mapsto w]$ satisfies all mandatory conjuncts of φ whose variables are bound by $b[x \mapsto w]$.*

The rows that satisfy the connection form a candidate set, since every witness satisfies it; the remaining conjuncts are tested in the body. For $\exists c$, the walk thus retrieves the customers in the nation of the bound supplier and checks $c.ck = o.ck$ on each. The order of candidates and of Boolean children affects the work but not the result. Our implementation visits first the child whose connection admits fewer candidates. Moreover, the outcome of a quantified subproblem depends on previously bound tuples only through the attribute values its body reads. These values key a decision store shared by all anchors.

Definition 4 (Key). *For a subformula ψ, let $\mathrm{FA}(\psi)$ be the attribute references $y.A$ occurring in ψ such that y is free in ψ, in a fixed order that every renaming of Definition 2 preserves. The key of ψ under b is $\kappa_\psi(b) = \big([\![y.A]\!]_b\big)_{y.A \in \mathrm{FA}(\psi)}$.*

The key covers every outer attribute read below the quantifier, not only by its connection. For example, $\mathrm{FA}(\exists s.\cdots) = \{\ell.sk, o.ck\}$, since $c.ck = o.ck$ two levels below reads the custkey of the order. The store maps a pair $([\varepsilon],k)$ to `good` if a witness exists and to `nogood` otherwise, where $[\varepsilon]$ is the class of $\varepsilon = \exists x.\varphi$ up to renaming. Both

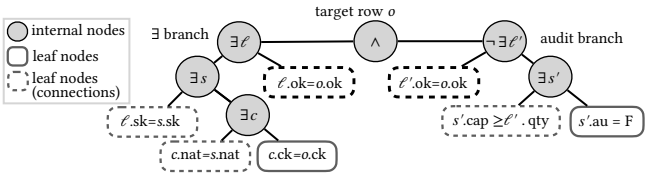


Figure 4: Decision graph of the running policy. Thick connections of the two branches are equal up to renaming ℓ' to ℓ, which Section 5 exploits; each branch binds its own lineitem.

$\exists x.\varphi$ and $\neg\exists x.\varphi$ are keyed by ε, and polarity is applied only on return. One entry thus serves every occurrence of ε under either polarity: a policy that requires $\exists x.\varphi$ in one branch and forbids an equal formula in another decides both from the same entry.

Algorithm and running example. Algorithm 1 walks the graph for each anchor. At an atom, it tests the comparison, and at a Boolean node, it short-circuits as soon as the value is determined. At a quantifier, it probes the store with $([\varepsilon],\kappa_\varepsilon(b))$. On a miss, an *expansion* tests the body on each candidate, stops at the first witness, and records the resulting outcome; later probes of the same pair skip the expansion.

Table 1 traces the walk. For o_1, the existential branch reads ℓ_1, s_1, and c_1, one candidate per quantifier, and finds its first witness. The audit branch reads both lineitems of o_1; for each, the only supplier with sufficient capacity is s_5, whose audit status is P. Hence $\exists\ell'$ records `nogood` at key 101, its negation holds, and o_1 is accepted. For o_2, the existential branch reaches $\exists s$ through ℓ_3 rather than ℓ_1, but with the same key (20,4), and reuses `good` without reading s_1 or c_1. The audit branch reads s_5 for ℓ_3 and then finds the forbidden witness (ℓ_4,s_3), so $\exists\ell'$ records `good` and o_2 is rejected. For o_3, no customer is in Ge, so the first probe of $\exists c$ at (Ge,9) reads nothing and records `nogood`; the second, reached through ℓ_6 and s_3 for the same anchor, hits this entry. The existential branch fails, and the audit branch is never entered.

Correctness. The correctness of the approach follows from the lemma and theorem below.

Lemma 1 (Coincidence). *Let ψ and ψ' be subformulas of a normal form of $\pi[\theta]$ that are equal up to a renaming of Definition 2, possibly $\psi = \psi'$, and let b and b' be assignments of their respective free variables. If $\kappa_\psi(b) = \kappa_{\psi'}(b')$, then $[\![\psi]\!]_b = [\![\psi']\!]_{b'}$.*

For $\psi = \psi'$, this is the coincidence lemma of first-order logic [18, Sec. 2.2], applied to attribute references. Since the renaming preserves relations and the order of FA, the general case follows by the same induction on ψ. The following theorem establishes the correctness; the proof is given in [1].

Theorem 1 (Correctness). *For every normal form of $\pi[\theta]$, order of children and candidates, and choice of candidate sets satisfying Definition 3, Algorithm 1 returns $\mathrm{acc}(T,\pi,\theta)$, and every stored entry $V[([\varepsilon],k)]$ equals $[\![\varepsilon']\!]_b$ for all $\varepsilon' \in [\varepsilon]$ and all b with $\kappa_{\varepsilon'}(b) = k$.*

Cost and limits. We count reads as in Figure 2. Each pair $([\varepsilon],k)$ is expanded at most once, and every later arrival costs one probe and no read. The reads are therefore bounded by the anchors plus the candidates of distinct keys rather than of every arrival, and short-circuiting keeps them well below this bound. On the running example, the walk reads 19 tuples, 14 of them distinct, and builds no intermediate tuple, whereas the plans of Figures 2b and 2c read 49 and 29 tuples and build 14 and 9, respectively. Four of the five

Algorithm 1 Decision-oriented policy evaluation

Require: instantiated policy $\pi[\theta]$ over target T, database $\mathcal{D}$
Ensure: $\mathrm{acc}(T,\pi,\theta)$

function EVALUATE($\pi[\theta]$,$\mathcal{D}$)
  $G \leftarrow$ decision graph of a normal form of $\pi[\theta]$
  $V \leftarrow \emptyset$ ▷ decision store, shared by all anchors
  return $\{r \in \mathrm{rids}(T) \mid \textsc{Decide}(\mathrm{root}(G), \{x_T \mapsto r\})\}$
function DECIDE(n,b)
  if n is an atom **then return** $[\![n]\!]_b$
  if $n = \bigodot_i n_i$ **then return** $\bigodot_i \textsc{Decide}(n_i, b)$ ▷ $\odot \in \{\wedge, \vee\}$, short-circuit
  let $n = Qx.\varphi$ with $Q \in \{\exists, \neg\exists\}$, and $\varepsilon = \exists x.\varphi$
  $k \leftarrow ([\varepsilon], \kappa_\varepsilon(b))$ ▷ Definition 4
  if $k \notin V$ **then** ▷ expansion
    $v \leftarrow$ nogood
    for all $w \in \mathrm{Cand}(x,\varphi,b)$ **do**
      if $\textsc{Decide}(\varphi, b[x \mapsto w])$ **then** $v \leftarrow$ good; **break**
    $V[k] \leftarrow v$
  return $V[k] = (Q = \exists \ ?\ \mathtt{good} : \mathtt{nogood})$

Table 1: Example walk with the ∃ branch first and candidate suppliers in ascending order of capacity. Each column group lists the probes of one order from top to bottom: the key, the witness outcome, and the candidates read by the expansion, or *hit* if the store answered the probe. Each anchor additionally reads its own tuple. In total: 16 probes, 2 hits, and 19 reads (3 anchors, 16 candidates) of 14 distinct tuples.

	o_1			o_2			o_3		
	Key		Reads	Key		Reads	Key		Reads
$\exists\ell$	101,4	good	ℓ_1	102,4	good	ℓ_3	103,9	nogood	ℓ_5,ℓ_6
$\exists s$	20,4	good	s_1	20,4	good	hit	21,9	nogood	s_2
$\exists c$	Fr,4	good	c_1				Ge,9	nogood	–
$\exists s$							22,9	nogood	s_3
$\exists c$							Ge,9	nogood	hit
$\neg\exists\ell'$	101	nogood	ℓ_1,ℓ_2	102	good	ℓ_3,ℓ_4			
$\exists s'$	65	nogood	s_5	70	nogood	s_5			
$\exists s'$	90	nogood	s_5	60	good	s_3			

repeated reads show what exact-key reuse misses. The lineitems ℓ_1 and ℓ_3 are read once by each branch (R3). The supplier s_5 is read for the quantities 65, 90, and 70, which never repeat, although by monotonicity the outcome at 65 decides the other two (R2). Moreover, the five probes of $\exists\ell$ and $\neg\exists\ell'$ carry the orderkey of the anchor and can never hit. With exact keys, the walk behaves like pipelined index nested-loop semijoins and antijoins with a subquery cache, such as PostgreSQL's Memoize, below every quantifier, a plan that optimizers choose only when their estimates favor it. Section 5 shows how relational structure lets the walk reuse decisions well beyond identical keys.

5 Rank-Based Execution and Outcome Reuse

In this section we first present the rank artifacts that MEMENTO builds offline independently of bindings. We then discuss how this makes the walk efficient and enables decision reuse beyond exact keys. Lastly, we present how we can share walks across polarities.

5.1 Rank Universes and Artifacts

A binding changes the operators and constants of a policy, but not which attributes its atoms compare. Hence, once the values of these attributes are sorted, every bound operator selects a point, a range, or the complement of a point. We materialize this order once, offline, as dense ranks.

Ranks. Attributes that atoms compare with each other, directly or transitively, form a *rank universe* and must share one type and collation; an attribute compared only with parameters, such as s_au, forms a universe of its own. For example, the running template induces six universes: one each for orderkeys, custkeys, suppkeys, nations, and audit statuses, and one that holds both quantities and capacities, which ω_6 compares (Figure 5(a)). Further, if $d_1 < \cdots < d_m$ are the distinct non-null values of a universe in $\mathcal{D}$, value d_i has *rank* i, and NULL has no rank (Section 2). As ranks preserve order, comparing two ranks under any $\omega \in \Omega$ gives the same outcome as comparing the values. Ranks are thus order-preserving dictionary codes [2, 9], which also make keys compact integers, e.g., the key (Fr,4) of $\exists c$ in Table 1 becomes (1,1).

From operators to rank ranges. We orient each connection as $x.A\,\omega\,\tau$, where x is the variable being bound and τ is a term over bound variables or a parameter. For a non-null value a of τ, let $\mathrm{lo}(a) = 1 + |\{i \mid d_i < a\}|$ and $\mathrm{hi}(a) = 1 + |\{i \mid d_i \le a\}|$, so that $\mathrm{hi}(a) = \mathrm{lo}(a)+1$ if a occurs in the universe, and $\mathrm{hi}(a) = \mathrm{lo}(a)$ otherwise, e.g., for a parameter absent from the data. Writing lo and hi for $\mathrm{lo}(a)$ and $\mathrm{hi}(a)$, the ranks of the values d with $d\,\omega\,a$ form at most two intervals $I_\omega(a)$:

ω	=	≠	<	≤	>	≥
$I_\omega(a)$	[lo,hi)	[1,lo)∪[hi,m+1)	[1,lo)	[1,hi)	[hi,m+1)	[lo,m+1)

Rank artifacts. For each referenced attribute $R.A$, the *forward index* stores the rank of A for every row of R, so that the walk obtains the rank of a bound attribute without reading its tuple. A null value is stored as 0, which no value has as its rank, since ranks start at 1; an atom that reads 0 is false. The *inverted index* $\mathrm{inv}_{R.A}$ stores the rows with non-null A sorted by rank, with an offset array, so that the rows with ranks in $[i,j)$ form one slice $\mathrm{inv}_{R.A}[i,j)$. For $a = [[\tau]]_b$, the slice $\mathrm{inv}_{\mathrm{rel}(x).A}[I_\omega(a)]$, empty if a is NULL, holds exactly the rows that satisfy the connection and is thus a candidate set (Definition 3); a quantifier entered through several equalities may use any of their slices. No artifact depends on the binding: a new operator, constant, Boolean form, or polarity changes only which slices the walk reads. A B-tree also supports range retrieval, but the offsets provide constant-time slice lengths, which the child order of Section 4.3 uses. The artifacts are rebuilt after updates; only the decision store is created per evaluation.

5.2 Reuse Beyond Exact Keys

We generalize the decision store of Section 4.3 so that the refinements below change only how decisions are keyed and looked up, not the walk. The walk accesses the store through three operations: each time it reaches ε under b, $\textsc{Key}(\varepsilon,b)$ forms the key k of the outcome of ε; $\textsc{Probe}(\varepsilon,k)$ returns good, nogood, or $\perp$ if no outcome is known; and $\textsc{Record}(\varepsilon,k,o)$ stores outcome o. The store of Section 4.3 is the special case $k = \kappa_\varepsilon(b)$. When KEY can tell that no row satisfies the connection, e.g., because the bound value is NULL, it returns $\emptyset$, and PROBE answers nogood without accessing the store. We refine this store in the following three ways:

(1) Core keys. If $\mathrm{FA}(\varepsilon)$ contains all attributes of a key K of $\mathrm{rel}(y)$ for a free variable y, the *core* $\mathrm{core}(\varepsilon)$ replaces the attributes of y by K. As K determines the row, two assignments agree on the core iff they agree on $\mathrm{FA}(\varepsilon)$, so the core narrows keys without changing which probes hit. It also exposes probes that never hit: if ε lies outside every quantifier, occurs once in the decision graph, and its core contains a key of T, no two anchors share its key, so we neither probe nor record ε. This holds for $\exists\ell$ and $\exists\ell'$, whose cores are $\{o.\mathrm{ok}\}$.

(2) Key-equality keys. If the connection of $\varepsilon = \exists x.(\gamma \wedge \varphi')$ is an equality $\gamma = (x.K = \tau)$ on a key K of $\mathrm{rel}(x)$, its slice holds at most one row w, and $[[\varepsilon]]_b = [[\varphi']]_{b[x \mapsto w]}$ depends only on $\kappa_{\varphi'}(b[x \mapsto w])$, the values that φ' reads from w and from earlier tuples (Lemma 1). Keyed by these values rather than by the value of τ, distinct rows share an outcome: $\exists s$ is keyed by (s.nat,o.ck), not (ℓ.sk,o.ck), so the suppliers s_2 and s_3 of o_3 share (Ge,9), which Definition 4 separates as (21,9) and (22,9). To form this key without rereading w, Key keeps *row facts*, the ranks that φ' reads from w, under the rank of $[[\tau]]_b$; only a value not seen before costs a lookup and a read of w. A value that is NULL, has no rank, or has an empty slice yields $\emptyset$, and an empty slice is recorded as a fact. On a miss, the expansion evaluates φ' on w alone. Thus o_2, reaching suppkey 20 again, forms (Fr,4) from the facts recorded when o_1 read s_1, and hits.

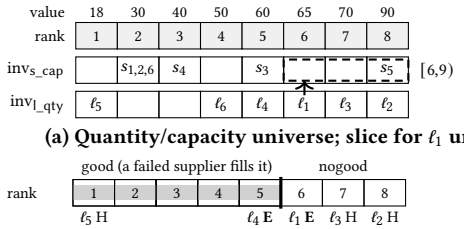
(3) Monotone outcomes. These refinements reuse an outcome only for a key expanded before. On an ordered connection, candidate sets are nested: for $s'.\mathrm{cap} \geq \ell'.\mathrm{qty}$, the candidates for a quantity of rank ρ, the suppliers of capacity rank at least ρ, shrink as ρ grows, so a witness for a quantity is one for every smaller quantity. Formally, let $\varepsilon = \exists x.\varphi$ have a connection $\gamma = (x.A\,\omega\,y.B)$ with $\omega \in \{<,\leq,>,\geq\}$, where y is free in ε and $y.B$ occurs in ε only in γ. This *ordered connection* splits the key of ε into the rank ρ of $y.B$ and the *residual key* $\kappa'_\varepsilon(b)$ over $\mathrm{FA}(\varepsilon) \setminus \{y.B\}$.

Lemma 2 (Monotone outcomes). *Let ε have an ordered connection with $\omega \in \{>,\geq\}$, and let b and b' agree on the residual key and give $y.B$ the ranks $\rho' \leq \rho$. If $[[\varepsilon]]_b = 1$, then $[[\varepsilon]]_{b'} = 1$. For $\omega \in \{<,\leq\}$, the same holds for $\rho' \geq \rho$.*

The lemma follows because a witness under b still satisfies the connection γ under b', and φ' does not read $y.B$; [1] gives the proof. For $\omega \in \{>,\geq\}$ and a fixed residual key, the good ranks thus form a prefix of the ranks $1,\ldots,m$ of the universe of $y.B$. For each pair (ε,κ'), we define an *envelope* that keeps the largest rank $\beta_\top$ recorded as good and the smallest rank $\beta_\bot$ recorded as nogood, initially 0 and $m+1$. Key returns (ρ,κ'); Probe answers good if $\rho \leq \beta_\top$, nogood if $\rho \geq \beta_\bot$, and $\bot$ in between; and Record raises $\beta_\top$ or lowers $\beta_\bot$. The answers are correct by Lemma 2, and as no expanded rank lies in the gap, an envelope answers every probe that an exact-key store answers. Each expansion narrows the gap, so a residual key costs at most one expansion per distinct rank probed in it, however often that rank is probed; two suffice in Figure 5(b). The case $\omega \in \{<,\leq\}$ mirrors the boundaries, and other subformulas keep exact keys. An envelope brackets the MIN or MAX aggregate into which quantified comparisons can be rewritten [19, 30], but lazily, only as tightly as probes require, and without a rewrite per bound operator.

5.3 Shared Walks across Branches

The refinements of Section 5.2 avoid repeating a subsearch, but branches still repeat the iterations they share. In Algorithm 1, Decide expands one quantifier at a time: for o_1, it iterates the lineitems of o_1 for $\exists\ell$, descending into $\exists s$ and $\exists c$, and then iterates them again for $\neg\exists\ell'$, descending into $\exists s'$, since both branches enter lineitem through $\ell.\mathrm{ok} = o.\mathrm{ok}$. Branches of opposite polarity cannot share a semijoin and an antijoin (Section 3; duplicate reads), but they can share this iteration: a *shared walk* iterates the slice once and evaluates both bodies on each lineitem, while each body still descends into its own nested quantifiers.



(a) Quantity/capacity universe; slice for ℓ_1 under $\geq$.

(b) Outcomes of $\exists s'$: two expansions (E), three hits (H).

Figure 5: Example rank artifacts and monotone outcomes. (a) Expanding $\exists s'$ for ℓ_1 under $\omega_6 \mapsto \geq$ reads $\mathrm{fwd}_{\mathbf{l_qty}}(\ell_1) = 6$ and the slice $\mathrm{inv}_{\mathbf{s_cap}}[6,9) = \{s_5\}$; under $\leq$, the same artifacts give $\mathrm{inv}_{\mathbf{s_cap}}[1,7)$. (b) Arrivals of the combined execution (Sec. 5.4): expanding rank 6 shows that all ranks ≥ 6 are nogood, and expanding rank 5 that all ranks ≤ 5 are good.

Algorithm 2 Walk of a bundle B of co-entered requests.

```
1:  function Walk(B,b)                                   ▷ witness outcomes of B, or ⊥
2:      k_j ← Key(ε_j,b) for j ∈ B                       ▷ resolves w for a key equality
3:      o_j ← Probe(ε_j,k_j) for j ∈ B;  P ← {j ∈ B | o_j = ⊥}
4:      Propagate(j) for j ∈ B\P; if the root is decided then return ⊥
5:      for all w ∈ Cand_B(b) while P ≠ ∅ do             ▷ one shared slice
6:          b' ← b[x ↦ w];  t_j ← [[φ_j]]^3_{b'} on the atoms of φ_j, for j ∈ P
7:          for all bundles B' required by {φ_j | j ∈ P, t_j = ⊥}, in plan order,
        recomputed do
8:              r ← Walk(B',b'); if r = ⊥ then return ⊥
9:              update t_j for j ∈ P with r, through the polarities of B'
10:         for j ∈ P with t_j = true: o_j ← good; Record(ε_j,k_j,good); P ←
        P\{j}; Propagate(j)
11:         if the root is decided then return ⊥          ▷ cancel
12:     for j ∈ P: o_j ← nogood; Record(ε_j,k_j,nogood)   ▷ slice exhausted
13:     return (o_j)_{j∈B}
```

Bundles. Quantifiers are *co-entered* if their connections are equal up to renaming and they are evaluated under the same assignment, i.e., both lie outside every quantifier, or their nearest enclosing quantifiers are co-entered. A *bundle* B is a set of co-entered quantifiers renamed to bind a common variable x. Under b, each member $Q_j x.\varphi_j$ poses a *request* for the witness outcome of $\varepsilon_j = \exists x.\varphi_j$, and all members share the slice $\mathrm{Cand}_B(b)$ of their common connection. In the running example, $\{\exists\ell, \neg\exists\ell'\}$ is a bundle, whereas $\exists s$ and $\exists s'$, entered through $\ell.\mathrm{sk} = s.\mathrm{sk}$ and $s'.\mathrm{cap} \geq \ell'.\mathrm{qty}$, form singleton bundles. A *traversal plan* partitions the quantifiers into bundles; our implementation bundles all co-entered quantifiers and orders their requests by the child order of Section 4.3. As co-entry is recursive, one walk serves requests of different branches at every depth.

Walking a bundle. Algorithm 2 replaces Decide. For a single request, it performs the probe and expansion of Algorithm 1 (lines 2–3, 5–10, and 12), and the caller applies the polarity (line 9). For a bundle, it answers the requests keyed $\emptyset$ or already stored, and iterates $\mathrm{Cand}_B(b)$ once for the remaining open requests P; for a key-equality bundle, $\mathrm{Cand}_B(b) = \{w\}$ was resolved by Key and is not read again. On each candidate, the walk evaluates every open body in Kleene's three-valued logic, with $\bot$ for quantifiers not yet decided: first its atoms (line 6), then the nested bundles that the undetermined bodies still require (lines 7–9). Each request closes under its own key and with its own polarity, so the walk shares reads but never fuses $\exists A \wedge \neg\exists B$ into $\exists(A \wedge \neg B)$.

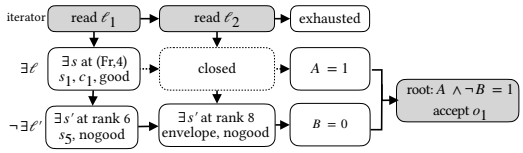


Figure 6: Shared walk of bundle $\{\exists \ell, \neg \exists \ell'\}$ for o_1: one iteration over the lineitems serves both requests without fusing them.

Interleaving changes how Boolean nodes short-circuit. DECIDE short-circuits a node when a child returns, but in a shared walk a request may close while its siblings are still iterating. PROPAGATE therefore pushes each closed outcome up through the requests and Boolean nodes suspended on the call stack, which may close them in turn, and once the root is decided, every walk on the stack returns $\perp$ before its next read (lines 4, 8, and 11). A cancelled request records nothing, since an incomplete iteration proves neither outcome, whereas outcomes already established stay in the store. EVALUATE runs the same loop on the root, with the anchor as the only candidate. For example, in Figure 6, the existential request of o_1 closes good at ℓ_1 with the witness (s_1, c_1). The root is not yet decided, so the iteration continues for the audit request, which finds no failed supplier at rank 6 and, through the envelope, none at rank 8. The slice is then exhausted, the audit request closes nogood, and propagation accepts o_1.

Correctness. Sharing a slice is sound as every bundled plan ensures that (i) $\text{Cand}_B(b)$ contains every witness of every request, as the common connection is a mandatory conjunct of every member (Definition 3); (ii) nested bundles bind fresh variables, so each outcome is a Boolean function of its local test and nested outcomes; and (iii) the Boolean nodes above and between bundles are those of $\pi[\theta]$.

THEOREM 2 (Correctness). *For any traversal plan with co-entered bundles, Algorithm 2 over rank artifacts and the decision store of Section 5.2 returns* $\text{acc}(T, \pi, \theta)$*, independently of evaluation order and of which store entries are skipped or evicted.*

The theorem combines the soundness of each extension. Rank slices are exactly the candidate sets of Definition 3, every answer of PROBE other than $\perp$ is correct by Lemmas 1 and 2, and (i)–(iii) make shared walks sound. A skipped or evicted entry only makes PROBE return $\perp$, which forces an expansion. [1] gives the full proof.

5.4 Putting It All Together

The last two extensions are complementary. Interleaving the branches forgoes the short-circuit between them: the walk of Section 4 never enters the audit branch of o_3, whereas a shared walk evaluates it at ℓ_5 and, without monotone outcomes, reads four suppliers there, for 19 reads in total (21 with neither decision store nor row facts). With them, rank 1 of ℓ_5 lies below the envelope boundary $\beta_\top = 5$, so the audit request is answered without a read and rejects o_3 before ℓ_6 is read. Exact and core keys gain little when a quantifier is entered through values that rarely repeat, monotone outcomes when residual keys are diverse, and shared walks when few quantifiers are co-entered; the walk then degrades to an index nested-loop semijoin or antijoin that stops at the first witness and builds no intermediate result.

6 Experimental Evaluation

We evaluate MEMENTO on MIMIC [29], TPC-H [57], and IMDB (JOB) [32] datasets and study (i) how it compares with existing enforcement mechanisms (Section 6.2); (ii) how multiplicity, parametricity, and width affect this comparison (Section 6.3); (iii) how it scales with the data (Section 6.4); and (iv) the space taken by its artifacts and decision store (Section 6.5). Overall, the advantage of MEMENTO grows with policy complexity, since more target rows and branches share each decision. It trails the best plan only when a selective filter hides most rows and each target row follows its own key path. Its time grows near-linearly with the data, and its artifacts take about as much space as the database's indexes.

6.1 Setup

Implementation. We implement MEMENTO as a PostgreSQL 16 extension: a C component hooks into the planner and the executor and loads the rank artifacts, and a C++ component implements the walk. For every access path of a protected relation, MEMENTO substitutes its own scan node, which applies the outcome of the walk as a bitmap over tuple locations (Figure 3). We refer to [1] for implementation details.

Platform and measurement. All experiments run on an Intel Xeon Gold 6430 server with 256 GB of RAM under Ubuntu 24.04. Every system runs single-threaded on the same pinned core with warm caches unless stated otherwise, and buffer pools and memory limits (160GB) keep the working set resident, so DuckDB never spills; PostgreSQL runs with JIT enabled and parallel workers disabled. We report, unless stated otherwise, the median of three timed runs after a warm-up. The timer covers planning and execution but, for every method, not data loading, index and artifact builds, or policy installation (Section 6.5 reports the cost of MEMENTO's artifacts). We set a timeout of 30 minutes for all methods.

Baselines. We consider six baselines that cover both ways policies are enforced today (Section 3): as a native row-level policy or as a rewrite of the user query. Each receives $\pi[\theta]$ as literal SQL and is optimized for it. This per-binding re-optimization exposes the bound operators and constants to the optimizer, keeps index access paths available, and thus favors the baselines. All methods share the same B-tree indexes, built ahead of time on all keys and on every attribute a policy compares: 33 on TPC-H, 69 on IMDB, and 82 on MIMIC. **PG-RLS** installs the policy with `CREATE POLICY` (Section 3). PostgreSQL 16 evaluates it as a security-barrier qualification, which limits how its optimizer may reorder and unnest it. **DBMS-O** is a commercial DBMS[2] whose fine-grained access control appends the predicate returned by a policy function to every query on the target. We flush its shared cache before every run, so no generated predicate or plan is reused. **Verbatim$_{\text{PG}}$** and **Verbatim$_{\text{Duck}}$** place $\pi[\theta]$ in the `WHERE` clause as correlated `EXISTS` and `NOT EXISTS` subqueries and leave unnesting to the optimizer. PostgreSQL 16 unnests only some subqueries, whereas DuckDB 1.5 (in memory) unnests arbitrary ones [37]. **Bespoke$_{\text{PG}}$** and **Bespoke$_{\text{Duck}}$** run, on the same two engines, a hand-written rewrite that decorrelates the policy into semijoins and antijoins [37] and turns a single ordered comparison into a `MIN` or `MAX` aggregate [19, 30]. We derived each

[2] Its license does not permit publishing benchmark results under its name.

Table 2: Datasets (left) and their policies by tier (right), 10 per tier. Rel. / col.: in the database, and in parentheses those the policies read. Univ.: rank universes. Const.: predicates with a constant. M to m: many to many.

dataset							predicates per policy			branches		policies of 10 with					
	rel. / col.	rows	target	univ.	tier	relations	join	ordered	const.	∃	¬∃	∨	shared	∃/¬∃	m to m	own key	kept, %
MIMIC IV demo	31 / 342 (21 / 71)	1.0M	chartevents 669K	35	simple	1.7 (1–2)	0.9	0.5	1.5	0.4	0.3	2	0	0	4	0	6.6 (<0.1–83)
					inter.	3.6 (3–4)	3.1	0.4	2.5	1.0	0.8	3	5	3	7	0	57 (1.7–96)
					complex	8.2 (6–10)	8.2	1.1	4.3	1.3	0.9	5	5	4	8	0	25 (2.8–90)
TPC-H (SF 10)	8 / 61 (8 / 39)	86.6M	lineitem 60.0M	24	simple	1.7 (1–2)	0.7	0.2	1.9	0.4	0.3	2	0	0	2	0	14 (0.1–82)
					inter.	4.1 (3–5)	3.0	0.2	1.9	1.0	0.8	4	5	3	3	0	22 (0.8–96)
					complex	7.3 (6–8)	7.1	0.4	2.7	1.1	1.0	4	7	4	6	0	9.2 (<0.1–91)
IMDB (JOB)	21 / 108 (19 / 43)	70.9M	title 2.5M	25	simple	2	1.0	0.0	5.6	1.0	0.0	0	0	0	0	5	18 (1.1–35)
					inter.	4.9 (4–5)	3.9	0.0	7.0	1.0	0.0	0	0	0	0	10	4.3 (<0.1–15)
					complex	11.2 (9–14)	10.2	0.0	12.6	1.0	0.0	0	0	0	0	10	0.2 (<0.1–4.2)

rewrite with the help of an LLM (Claude-Opus-5.5), then inspected it and validated its result. Bespoke is deliberately strong: it applies rewrites that neither optimizer derives on its own, at a per-policy expert effort that a deployment cannot repeat for every binding, since rebinding an operator can invalidate the rewrite ($\geq$ maps to `MAX`, $\leq$ to `MIN`, and $=$ to neither). The DuckDB baselines also have an advantage independent of enforcement: DuckDB is an in-memory columnar engine with vectorized hash joins, whereas Memento and the other baselines run in PostgreSQL. **Memento** runs with the same settings as PG-RLS.

Datasets and workloads. Table 2 summarizes the three datasets and workloads: **TPC-H** (SF 10) by default, the skewed **IMDB** data of JOB [32], and the **MIMIC-IV demo**. A workload is a triple (Q_T,π,θ) of a query, a policy template, and a binding; we use 10 per tier on each dataset, 90 in total. The query Q_T is `SELECT count(*) FROM Table` over the protected target `Table` of the dataset. It has no predicate of its own, so enforcement alone determines its cost, and every method answers the same query.

For TPC-H and MIMIC, we design 30 templates each under rules fixed before any run. Each template belongs to a role, states why the role may see a row, and takes the role's attributes as parameters. It stems from a benchmark query (e.g., TPC-H Q5: customers who bought from a supplier in their own nation) or from a regulation (e.g., a clinician sees only the patients they treat). Tiers count the relations a template reads, the target included: *simple* (1–2), *intermediate* (3–5), and *complex* (≥ 6). Each tier mixes, in proportions fixed in advance, key joins, ordered comparisons of two attributes, templates with little to reuse, many-to-many connections with a meaning (same nation, admission, or ward), and $\neg\exists$ branches.

For the JOB dataset, we derive the templates on `title` from its queries, one per family, so that the workloads include templates we did not write. We keep each query's joins, drop its `LIKE` and `IS NULL` filters, which our policy model excludes, so that a template allows at least the titles of its query, and express `IN` and `BETWEEN` with $=$, $\leq$, and $\geq$. We exclude 2c and 32a, which allow no title [60]. The 10 queries with the most relations form the complex tier. Others, cut to 5 relations, form the intermediate tier, and cut to 2 relations with a filter on the second, the simple tier. A cut keeps whole paths from `title` to the filtered relations and carries its size as a subscript: $13b_2$ is simple, $13b_5$ intermediate, and 13b the full query.

In the default workloads, each template has the one binding its role states. Each ω_i is the role's comparison, e.g., $\geq$ in $s.\text{acctbal} \geq \ell.\text{extendedprice}$ for a supplier that could finance an item from its balance (T14, T28), and each constant is a value of the role, such as a nation. The parametricity experiments below vary the bindings. No default template nests one `EXISTS` inside another; the width experiments below do. T01–T30 (TPC-H) and M01–M30 (MIMIC) are numbered by tier: 01–10 simple, 11–20 intermediate, 21–30 complex. [1] lists each template with its role, rationale, and SQL.

6.2 Comparison with Existing Enforcement

Overall. Figure 7 shows the speedup of Memento over each baseline on all 90 workloads. Against every baseline, the median speedup is largest on complex policies. There, Memento is faster than PG-RLS, DBMS-O, $\text{Verbatim}_{\text{PG}}$, and $\text{Bespoke}_{\text{PG}}$ on 29 or 30 of 30 (medians 58×, 31×, 32×, and 13×), and than $\text{Verbatim}_{\text{Duck}}$ and $\text{Bespoke}_{\text{Duck}}$ on 25 and 21 (3.8× and 2.2×). The walk pays once per target row and once per distinct decision. The shapes that let one decision serve many rows, namely joins shared by two branches and many-to-many connections, grow more frequent with the tier (Table 2). On complex TPC-H policies, the walk answers a median of 62M lookups from 2,473 decisions. Memento loses where a policy leaves nothing to reuse. We discuss the baselines by the source of wasted work (Section 3) that dominates their cost. Two policies recur: T30, which allows the urgent lineitems of one brand whose customer and supplier belong to America, and T28, which adds by $\vee$ a price comparison with the balance of an EU supplier.

Native enforcement. PG-RLS and DBMS-O evaluate the policy as written and repeat the witness search. On TPC-H, PG-RLS turns none of the 46 branches into a join and reruns 15 of them for every lineitem. It is the slowest PostgreSQL method on 55 of the 90 workloads and times out on 6. On T30, PG-RLS takes 62.9 s and DBMS-O 93.6 s, and Memento 2.3 s. Memento reaches the order, customer, and supplier by keys, and its decisions sit at pairs of nations, at most 625 for 60M lineitems. On T28, both baselines time out, and Memento takes 4.4 s. The native baselines win only 3 (PG-RLS) and 5 (DBMS-O) workloads, mostly simple policies with little to search, such as MIMIC M02, which keeps 0.04% of the charts.

Rewriting in PostgreSQL. PostgreSQL unnests 28 of the 46 correlated TPC-H branches of $\text{Verbatim}_{\text{PG}}$ itself, but an unnested branch can still repeat its search. On T30 and two other policies, it becomes a nested-loop semijoin that looks up the order, customer, and nation again for every lineitem. On complex TPC-H policies, $\text{Verbatim}_{\text{PG}}$ reads a median of 233M base tuples, against 80M for the walk. Unnesting by hand ($\text{Bespoke}_{\text{PG}}$) removes the repeated search but pays the witness blowup. On the same policies, it builds a median of 175M intermediate tuples, whereas the walk builds none. On T30, $\text{Bespoke}_{\text{PG}}$ first collects the distinct order–supplier pairs of all 60M lineitems and is 5.7× slower than $\text{Verbatim}_{\text{PG}}$. On T22, whose two branches build small sets, it is 33× faster. Against the

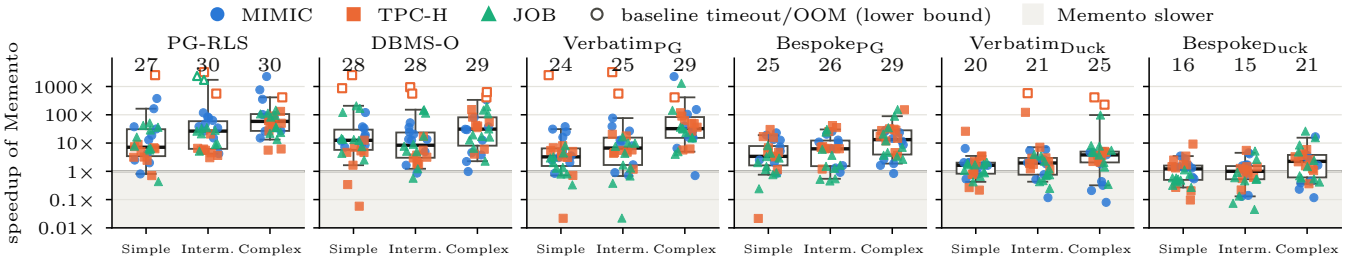


Figure 7: Speedup of MEMENTO over each baseline on all 90 workloads (log scale; per tier, 10 per dataset). Boxes span quartiles and whiskers the 5th–95th percentiles. The number above each box counts the workloads (of 30) on which MEMENTO is faster. Hollow markers: the baseline timed out (30 min) or ran out of memory, so the speedup is a lower bound.

better of the two, MEMENTO wins on 24, 25, and 29 of 30 policies, with medians of 3.3×, 5.6×, and 11.9×. PostgreSQL wins clearly where an index reaches a selective filter and skips most target rows, while MEMENTO still passes over all of them. On T10, which allows the lineitems of one brand and container, it fetches them through the part key in 48 ms, against 2.2 s for MEMENTO. On JOB 1a, which allows the titles ranked in the top 250, it takes 9 ms, against 406 ms.

Rewriting in DuckDB. Verbatim$_{\text{Duck}}$ unnests every subquery and thus trades repeated search for the witness blowup. On T28 and two other policies, it unnests a branch into a join of every order with every supplier of its customer's nation, 60 billion pairs by its own estimate: the two others exceed 30 minutes, and T28 exhausts the 160 GB memory limit. On T14, T28's price comparison alone, Verbatim$_{\text{Duck}}$ takes 68.2 s, whereas MEMENTO keeps one boundary for all lineitems (Lemma 2) and takes 0.56 s. Bespoke$_{\text{Duck}}$ runs our hand-unnested query on the vectorized engine and is the strongest baseline: MEMENTO is faster on 16, 15, and 21 of 30 policies, with medians of 1.2×, 1.0×, and 2.2×. MEMENTO wins where branches meet on a value that many target rows share: by 5.9× on T30, by 5.5× on T13, which closes the same cycle over four relations, by 4.2× on T28, and by 17× on MIMIC M23, where every chart of an ICU stay reaches one decision. On JOB 30c, it wins by 8.5×, since Bespoke$_{\text{Duck}}$ joins all 36M rows of cast_info, whereas the walk stops at each title's first witness. Of the 38 losses, 21 are on JOB, where each policy is a single ∃ branch of key joins entered at the key of title, so each title brings its own value. Bespoke$_{\text{Duck}}$ also wins by up to 3.7× on three TPC-H policies where both methods do the same work and its batches make that work cheaper; by 10× on T05, a ¬∃ on the two-column key of partsupp, which MEMENTO enters on the part alone, reading about 3.5 rows per lineitem; and by 7–9× on M14, M19, and M28, which tie each chart to the time of a transfer, a lab test, or an input, so few decisions repeat. Verbatim$_{\text{Duck}}$ evaluates a subquery once per distinct value, so it beats Bespoke$_{\text{Duck}}$ on 19 policies, such as M23 (35 vs. 152 ms), but trails it by a median of 2.0× on TPC-H, where each lineitem brings its own values. Against the faster DuckDB form, MEMENTO wins on 14, 15, and 21 of 30. On four of the nine complex losses to Bespoke$_{\text{Duck}}$, MEMENTO still beats Verbatim$_{\text{Duck}}$; there, the gap is our hand unnesting, which a deployment would have to redo for every policy and binding.

6.3 Controlled Experiments

Setup. In these experiments, we vary one property of a policy or its data and target one requirement of Section 3. *Multiplicity* varies how many target rows share one decision, testing reuse beyond

Table 3: Multiplicity policies, which keep a customer C when some supplier S satisfies them, and their sizes for each k.

Group: $S.g_k = C.g_k \wedge S.g_5 = 0$			**Balance:** $S.g_k = C.g_k \wedge C.bal \le S.bal \le \theta_k$		
groups k	5	25	250	2,500	25,000
suppliers per customer	20,000	4,000	400	40	4
θ_k of Balance	1,202	1,204	1,227	1,456	3,047
pairs a plan joins	30B	6B	600M	60M	6M

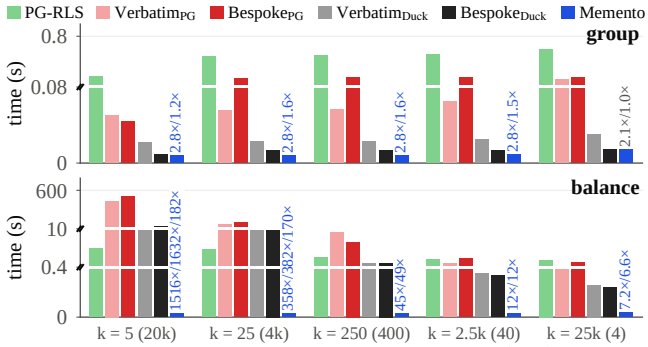


Figure 8: Impact of multiplicity on the Group (top) and Balance (bottom) policies; k is the number of groups, with the suppliers per customer in parentheses. Numbers on blue bars show speedup against best Verbatim/Bespoke baselines.

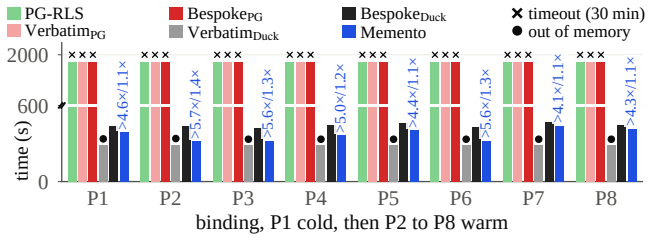


Figure 9: Parametricity experiments. Numbers on blue bars show speedup against best Verbatim/Bespoke baselines.

exact keys (R2). *Parametricity* rebinds the operators, constants, polarity, and Boolean form of one template, which changes the query of every plan but not the ranks on which decisions are kept (R4). *Width* varies the number of branches, which adds subqueries to a plan but only requests to a shared walk (R3).

Multiplicity. We add a column that splits the 1.5M customers and 100,000 suppliers into k groups, so that a customer reaches the $100{,}000/k$ suppliers of its group, and vary k from 5 to 25,000. Table 3 lists the two policies and their sizes. *Group* keeps a customer when its group has a supplier in a fixed fifth of the groups, so the decision depends on the group alone. *Balance* keeps a customer when its

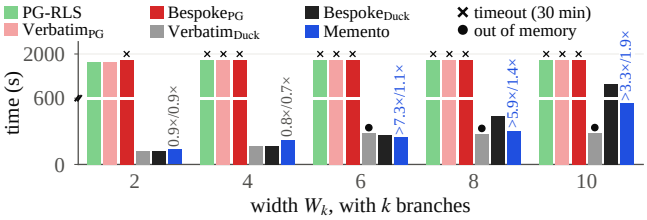


Figure 10: Impact of width. Numbers on blue bars show speedup against best Verbatim/Bespoke baselines.

group has a supplier whose balance lies between the customer's and θ_k, which we set so that both policies keep 20% of the customers. This comparison reads the target, so the decision depends on the customer and not only on its group. It is ordered on purpose, since a plan could hash an equality together with the group. Figure 8 shows that multiplicity alone costs no method much: under Group, every method stays under a second, since a plan reduces the suppliers to the groups that pass and the walk decides each group once. Under Balance, this reduction is no longer possible, and the unnested plans join each customer with every supplier of its group, 30 billion pairs at $k=5$. Bespoke$_{PG}$ then takes 506 s and Bespoke$_{Duck}$ 45.7 s. PG-RLS, otherwise usually the slowest method, takes 5.1 s, since its per-customer subquery stops at the first qualifying supplier. Memento also stops at the first witness, but additionally keeps per group the envelope of Section 5.2, which decides later customers of the group with one comparison (Lemma 2). It takes 28 ms at $k=5$ and 36 ms at $k=25{,}000$, 1632× and 6.6× faster than Bespoke$_{Duck}$. The fewer the groups, the more customers share an envelope, so the gap is widest where the store is designed to help.

Parametricity. We use one procurement role on lineitem with four policies of two branches each. Every branch nests three quantifiers: the outermost binds the order, customer, and part of a line by their keys, and below it, in either order, sit an $\exists$ or $\neg\exists$ on partsupp with its supplier and a quantifier over a scope of the branch's own. Bindings P1 to P8 change the Boolean combination of the policies, the combination and polarity of the branches, the constants, and the operators. They alter 13 to 22 of the 128 predicates of P1 and keep 34% to 67% of the lineitems. Each method runs all eight bindings in one session, P1 cold, with nothing cleared in between. Figure 9 shows the results. Verbatim$_{Duck}$ exhibits the witness blowup (Section 3): where a branch's scope encloses the partsupp, it carries every pair of a line and a scope row, and fills 160 GB on all eight bindings. Verbatim$_{PG}$ exhibits the repeated search (Section 3): it reruns the branches for every line, staying under 14 GB but reaching the 30-minute cap on all eight. Bespoke$_{Duck}$ avoids both but cannot use the Boolean form to save work. It computes each branch over all lines, joins the results to lineitem, and only then applies the form, so it peaks at 64 GB to 66 GB whether a binding keeps 34% or 67%. Its plans are also tied to one binding, so without a result cache DuckDB recomputes even the 16 of 64 branches that repeat exactly across bindings. Memento keeps no decisions across bindings either, but its decisions sit at ranks rather than at operators or constants, so every binding fills them at the same values from the same artifacts. A binding only sets how far each walk goes, since its connectives determine how many branches a line still needs: 6.4 to 8.4 reads per line, at a peak of 32 GB.

Width. The target is lineitem, and each branch again nests three quantifiers. W_k combines k branches by $\vee$, $\wedge$, and $\neg$, for k from 2 to 10: from 6 quantifiers and 32 predicates at W_2 to 30 and 160 at W_{10}, with 1 to 7 $\neg\exists$. Every width keeps about 60% of the lineitems. We run each width once cold and once warm in a new session, and report the warm run. Width changes what a plan must build far more than what the walk must do. Each branch of Bespoke$_{Duck}$ adds a subquery with its own joins, joined to lineitem, so from W_2 to W_{10} its peak grows from 38 GB to 78 GB and its time 8.9×. In Memento, a new branch is only another request on a walk already under way, since branches co-entered through the same connection share one walk (Section 5.3). Its rows read grow 2.2×, from 214M to 463M, and its peak from 22 GB to 31 GB. As Figure 10 shows, Memento trails Bespoke$_{Duck}$ at W_2 and W_4 and overtakes it from W_6: with few branches, a plan has few subqueries to compute, whereas the walk visits all 60M lines whatever the width. The step from W_8 to W_{10} comes from a branch that rereads lineitem through its supplier. A supplier has about 600 lines, and the branch's tests on two dates and by $\neq$ rule out few of them, so hundreds of witnesses remain at each supplier and every later arrival compares with them. This work is comparisons rather than reads, so here time grows much faster than reads. Verbatim$_{Duck}$ follows Bespoke$_{Duck}$ on W_2 and exhausts its 160 GB memory limit from W_4 on.

6.4 Scalability

Figure 11 runs the TPC-H policies at scale factors 0.1, 1, 10, and 100, with artifacts built for each scale. The time of Memento follows the number of lineitems, since the walk visits each lineitem once and decides what lies behind it once per value. With every tenfold step in scale, the tier medians grow 10× to 15×. Per lineitem, the simple, intermediate, and complex medians cost 23, 51, and 55 ns at SF 10, and 22, 59, and 73 ns at SF 100, where the complex median takes 43.9 s. A complex policy joins a median of 7.5 relations against 4 for an intermediate one, yet costs about as much per lineitem: its further relations are reached by keys and decided once per value.

6.5 Space Overhead

Artifacts. Memento requires a forward index and an inverted index per rank universe (recall Section 5.1). In addition, it requires a dictionary of its values and per relation and row ids. The following table shows this for 30 templates of each dataset.

Dataset	Forward	Inverted	Dictionary	Others	Total
MIMIC	36.9 MB	38.2 MB	2.8 MB	13.7 MB	91.6 MB
TPC-H	3.43 GB	3.87 GB	0.39 GB	0.69 GB	8.39 GB
JOB	0.94 GB	1.14 GB	0.61 GB	0.57 GB	3.25 GB

This space is about the same as the database's own B-tree indexes that baselines use (e.g., ≈8.0 GB on TPC-H), and they are built single-threaded in 24 minutes. They grow with rows, never with policy bindings. Lineitem, for instance, with 60M of the 87M rows in TPC-H accounts for 76% of the artifacts, and each of its 12 columns costs 5–7% of its table. Only the dictionaries grow with distinct values, and only those of the 15M orderkeys and the order totals are large.

Decision store. The decision store lives for one query. Most of it is the arrays kept per rank of a key column (Section 5.2), so it grows with the ranks the walk reaches, not with the rows it reads. It takes

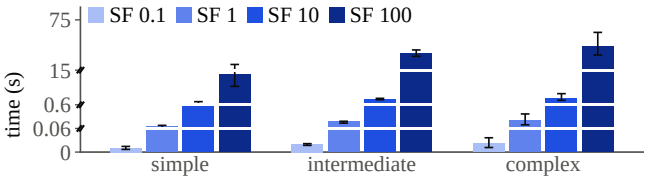


Figure 11: Scalability on TPC-H (median time per tier).

a median of 81.5 MB on TPC-H, about 1.4 bytes per lineitem, and at most 277.4 MB on T27. On MIMIC and IMDB, it stays under 5.4 MB.

7 Related Work

Fine-grained access control. Since query modification [52], content-based policies have been enforced by rewriting the user query or restricting what it sees, as in the Truman and non-Truman models [48] and Hippocratic databases [3, 31]. Production engines attach the policy predicate to every scan of the target [20, 34, 43, 50, 54], policy languages [13, 41] and middleware [21, 33, 56] translate a policy into such a predicate, and Sieve [45, 49] caches these rewrites per querier for disjunctive allow rules without ¬∃. All of them fix the predicate before evaluation and hand it to the engine as a query. Other systems cache decisions: Blockaid [63] caches a solver's verdict per query shape, and the relationship-based Zanzibar [44] decides one request at a time from cached subresults of its rules. None admits operators bound at query time, and no cache decides the target rows of one evaluation.

Evaluating quantified subqueries. Once a policy is a query, its cost depends on how the engine evaluates `EXISTS` and `NOT EXISTS`. DuckDB [46] and Umbra [36] unnest subqueries [35, 37] by pushing a dependent join down until the subquery no longer reads the outer query, and evaluate it once per distinct outer binding. Mark joins [38] keep a quantified answer as a Boolean column, Elhemali et al. [17] choose among execution strategies, and Rao and Ross [47] reuse a subquery's invariant part across outer rows. Yannakakis-style reduction [6, 59, 60, 62], factorized evaluation [40, 58], predicate transfer and bitmap filters [61, 64], information passing [5, 28, 51, 65], and diamond-hardened and TreeTracker joins [10, 24] shrink, compress, or prune join intermediates. Oracle coalesces an `EXISTS` and a `NOT EXISTS` into one scan when they differ only in their filters [7]. Parametric query optimization prepares plans for regions of constants [12, 25, 27], and adaptive processing changes a plan during execution [16]. Each technique makes a tuple-producing plan cheaper, but none removes all three sources of waste of Section 3, and the plan stays tied to the operators it was optimized for. Our walk builds no intermediate tuples and stops at the first witness. It shares one iteration across branches of opposite polarity even when they diverge into different relations below the shared connection, unlike coalescing. Its rank slices let one strategy and one set of artifacts serve every operator and constant.

Memoization and decision reuse. Subquery caches, such as PostgreSQL's Memoize and Oracle's scalar subquery cache, reuse a result per exact correlation key, as Hellerstein and Naughton [23] do for expensive predicates. AND/OR search with context-based caching [15] and tabled logic programs [11] memoize a subproblem under the values it depends on, and tabling answers a negated subgoal from the table of its positive counterpart. Our decision store carries these ideas to relational witnesses. Its key holds the attribute values that cross into a quantifier, narrowed through a key join to the values the body reads from the joined row, rather than tuple identities or the correlation key, and one polarity-independent outcome serves an existential branch and its negation. Beyond exact keys, semantic caching [14] and subsumptive tabling [53] answer a query from a cached one that contains it, and rewriting a quantified comparison into a `MIN` or `MAX` aggregate [19, 30] decides all keys with one value. Our envelopes reach this effect lazily and without a rewrite, which a rebinding would invalidate (R2): they keep two ranks per residual key, tighten them only as far as probes require, and answer a probe with one comparison. Our ranks are order-preserving dictionary codes [9], used for binding independence and compact keys rather than for compression.

8 Conclusion

We studied the enforcement of parametric content-based row-level policies, whose operators and constants are bound only at query time and whose branches mix the existence and absence of witnesses. Prior enforcement reduces such a policy to a query, prior query processing makes that query cheaper by decorrelating, reducing, or sharing the tuples it produces, and prior caching reuses a result for a repeated argument. We instead treat enforcement as a decision problem. Memento compiles an instantiated policy into a decision graph, decides each target row by a depth-first walk that stops at the first witness, and reuses each witness outcome across target rows, across keys on the same side of an ordered threshold, and across branches of opposite polarity. Dense, order-preserving ranks let one set of artifacts serve every binding. To our knowledge, no prior mechanism enforces such policies or reuses witness outcomes in these ways. Memento outperforms native row-level security on complex policies with median speedups of 58× (PostgreSQL) and 31× (commercial DBMS), and beats a hand-tuned DuckDB rewrite on 21 of 30 complex policies. Future work includes maintaining the rank artifacts incrementally under updates, combining the walk with index access when a selective filter skips most target rows, and extending the model to aggregation within quantifiers.

References

[1] 2026. https://github.com/IITD-data-systems/parametric-cbac.

[2] Daniel J. Abadi, Samuel Madden, and Miguel Ferreira. 2006. Integrating Compression and Execution in Column-Oriented Database Systems. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 671–682. doi:10.1145/1142473.1142548

[3] Rakesh Agrawal, Jerry Kiernan, Ramakrishnan Srikant, and Yirong Xu. 2002. Hippocratic Databases. In *Proceedings of the 28th International Conference on Very Large Data Bases (VLDB)*. VLDB Endowment, Hong Kong, China, 143–154. doi:10.1016/B978-155860869-6/50021-4

[4] Amazon Web Services. n.d.. AWS Identity and Access Management: Policy Evaluation Logic. AWS Identity and Access Management User Guide. https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html Accessed 2026-10-30.

[5] François Bancilhon, David Maier, Yehoshua Sagiv, and Jeffrey D. Ullman. 1986. Magic Sets and Other Strange Ways to Implement Logic Programs. In *Proceedings of the Fifth ACM SIGACT-SIGMOD Symposium on Principles of Database Systems (PODS)*. 1–15. doi:10.1145/6012.15399

[6] Liese Bekkers, Frank Neven, Stijn Vansummeren, and Yisu Remy Wang. 2025. Instance-Optimal Acyclic Join Processing Without Regret: Engineering the Yannakakis Algorithm in Column Stores. *Proceedings of the VLDB Endowment* 18, 8 (2025), 2413–2426. doi:10.14778/3742728.3742737

[7] Srikanth Bellamkonda, Rafi Ahmed, Andrew Witkowski, Angela Amor, Mohamed Zaït, and Chun Chieh Lin. 2009. Enhanced Subquery Optimizations in Oracle. *Proceedings of the VLDB Endowment* 2, 2 (2009), 1366–1377. doi:10.14778/1687553.1687563

[8] Elisa Bertino and Ravi Sandhu. 2005. Database Security—Concepts, Approaches, and Challenges. *IEEE Transactions on Dependable and Secure Computing* 2, 1 (2005), 2–19. doi:10.1109/TDSC.2005.9

[9] Carsten Binnig, Stefan Hildenbrand, and Franz Färber. 2009. Dictionary-Based Order-Preserving String Compression for Main Memory Column Stores. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 283–296. doi:10.1145/1559845.1559877

[10] Altan Birler, Alfons Kemper, and Thomas Neumann. 2024. Robust Join Processing with Diamond Hardened Joins. *Proceedings of the VLDB Endowment* 17, 11 (2024), 3215–3228. doi:10.14778/3681954.3681995

[11] Weidong Chen and David S. Warren. 1996. Tabled Evaluation with Delaying for General Logic Programs. *J. ACM* 43, 1 (1996), 20–74. doi:10.1145/227595.227597

[12] Richard L. Cole and Goetz Graefe. 1994. Optimization of Dynamic Query Evaluation Plans. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 150–160. doi:10.1145/191839.191872

[13] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew M. Wells. 2024. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1, Article 118 (2024), 670–697 pages. doi:10.1145/3649835

[14] Shaul Dar, Michael J. Franklin, Björn Thór Jónsson, Divesh Srivastava, and Michael Tan. 1996. Semantic Data Caching and Replacement. In *Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB)*. 330–341.

[15] Rina Dechter and Robert Mateescu. 2007. AND/OR Search Spaces for Graphical Models. *Artificial Intelligence* 171, 2-3 (2007), 73–106. doi:10.1016/j.artint.2006.11.003

[16] Amol Deshpande, Zachary Ives, and Vijayshankar Raman. 2007. Adaptive Query Processing. *Foundations and Trends in Databases* 1, 1 (2007), 1–140. doi:10.1561/1900000001

[17] Mostafa Elhemali, César A. Galindo-Legaria, Torsten Grabs, and Milind M. Joshi. 2007. Execution Strategies for SQL Subqueries. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 993–1004. doi:10.1145/1247480.1247598

[18] Herbert B. Enderton. 2001. *A Mathematical Introduction to Logic* (2nd ed.). Harcourt/Academic Press, San Diego, CA, USA.

[19] Richard A. Ganski and Harry K. T. Wong. 1987. Optimization of Nested SQL Queries Revisited. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 23–33. doi:10.1145/38713.38723

[20] Google Cloud. n.d.. Introduction to BigQuery Row-Level Security. https://cloud.google.com/bigquery/docs/row-level-security-intro. Accessed 2026-10-26.

[21] Hasura Inc. n.d.. Hasura GraphQL Engine: Configure Row Permissions. Hasura GraphQL Engine v2.x Documentation. https://hasura.io/docs/2.0/auth/authorization/permissions/row-level-permissions/ Accessed 2026-10-30.

[22] HCL Software. n.d.. How Security Labels Control Access. HCL OneDB (Informix) Security Guide. https://help.hcl-software.com/onedb/current/sec/ids_lb_022.html Accessed 2026-10-26.

[23] Joseph M. Hellerstein and Jeffrey F. Naughton. 1996. Query Execution Techniques for Caching Expensive Methods. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*. 423–434. doi:10.1145/233269.233359

[24] Zeyuan Hu, Yisu Remy Wang, and Daniel P. Miranker. 2026. TreeTracker Join: Simple, Optimal, Fast. *ACM Transactions on Database Systems* 51, 2, Article 12 (2026), 26 pages. doi:10.1145/3774325

[25] Arvind Hulgeri and S. Sudarshan. 2002. Parametric Query Optimization for Linear and Piecewise Linear Cost Functions. In *Proceedings of the 28th International Conference on Very Large Data Bases (VLDB)*. 167–178. doi:10.1016/B978-155860869-6/50023-8

[26] IBM. n.d.. GRANT (exemption) statement. IBM Db2 SQL Reference. https://www.ibm.com/docs/en/db2/11.5.x?topic=statements-grant-exemption Accessed 2026-10-26.

[27] Yannis E. Ioannidis, Raymond T. Ng, Kyuseok Shim, and Timos K. Sellis. 1992. Parametric Query Optimization. In *Proceedings of the 18th International Conference on Very Large Data Bases (VLDB)*. 103–114.

[28] Zachary G. Ives and Nicholas E. Taylor. 2008. Sideways Information Passing for Push-Style Query Processing. In *Proceedings of the 24th International Conference on Data Engineering (ICDE)*. 774–783. doi:10.1109/ICDE.2008.4497486

[29] Alistair Johnson, Lucas Bulgarelli, Tom Pollard, Steven Horng, Leo Anthony Celi, and Roger Mark. 2023. MIMIC-IV Clinical Database Demo (version 2.2). PhysioNet. doi:10.13026/dp1f-ex47

[30] Won Kim. 1982. On Optimizing an SQL-like Nested Query. *ACM Transactions on Database Systems* 7, 3 (1982), 443–469. doi:10.1145/319732.319745

[31] Kristen LeFevre, Rakesh Agrawal, Vuk Ercegovac, Raghu Ramakrishnan, Yirong Xu, and David J. DeWitt. 2004. Limiting Disclosure in Hippocratic Databases. In *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, Toronto, Canada, 108–119. doi:10.1016/B978-012088469-8.50013-9

[32] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215. doi:10.14778/2850583.2850594

[33] Aastha Mehta, Eslam Elnikety, Katura Harvey, Deepak Garg, and Peter Druschel. 2017. Qapla: Policy Compliance for Database-Backed Systems. In *26th USENIX Security Symposium (USENIX Security)*. 1463–1479. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/mehta

[34] Microsoft. n.d.. Row-Level Security (SQL Server and Azure SQL). https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security. Accessed 2026-10-26.

[35] Thomas Neumann. 2025. Improving Unnesting of Complex Queries. In *Datenbanksysteme für Business, Technologie und Web (BTW)*. 25–47. doi:10.18420/BTW2025-01

[36] Thomas Neumann and Michael J. Freitag. 2020. Umbra: A Disk-Based System with In-Memory Performance. In *10th Conference on Innovative Data Systems Research (CIDR)*.

[37] Thomas Neumann and Alfons Kemper. 2015. Unnesting Arbitrary Queries. In *Datenbanksysteme für Business, Technologie und Web (BTW) (Lecture Notes in Informatics, Vol. P-241)*. Gesellschaft für Informatik, Hamburg, Germany, 383–402.

[38] Thomas Neumann, Viktor Leis, and Alfons Kemper. 2017. The Complete Story of Joins (in HyPer). In *Datenbanksysteme für Business, Technologie und Web (BTW)*. 31–50.

[39] OASIS Open. 2013. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard. https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.pdf 22 January 2013.

[40] Dan Olteanu and Maximilian Schleich. 2016. Factorized Databases. *SIGMOD Record* 45, 2 (2016), 5–16. doi:10.1145/3003665.3003667

[41] Open Policy Agent. n.d.. Open Policy Agent (OPA) Documentation. https://www.openpolicyagent.org/docs. Accessed 2026-10-26.

[42] Oracle Corporation. n.d.. Enforcing Row-Level Security with Oracle Label Security. Oracle Database 2 Day + Security Guide, 12c Release 2. https://docs.oracle.com/en/database/oracle/oracle-database/12.2/tdpsg/enforcing-row-level-security-with-oracle-label-security.html Accessed 2026-10-26.

[43] Oracle Corporation. n.d.. Using Oracle Virtual Private Database to Control Data Access. Oracle Database Security Guide, Oracle Database 19c. https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/using-oracle-vpd-to-control-data-access.html Oracle Database 19c; accessed 2026-10-30.

[44] Ruoming Pang, Ramón Cáceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. 2019. Zanzibar: Google's Consistent, Global Authorization System. In *USENIX Annual Technical Conference (ATC)*. 33–46.

[45] Primal Pappachan, Roberto Yus, Sharad Mehrotra, and Johann-Christoph Freytag. 2020. Sieve: A Middleware Approach to Scalable Access Control for Database Management Systems. *Proceedings of the VLDB Endowment* 13, 11 (2020), 2424–2437. doi:10.14778/3407790.3407835

[46] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1981–1984. doi:10.1145/3299869.3320212

[47] Jun Rao and Kenneth A. Ross. 1998. Reusing Invariants: A New Strategy for Correlated Queries. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 37–48.

[48] Shariq Rizvi, Alberto O. Mendelzon, S. Sudarshan, and Prasan Roy. 2004. Extending Query Rewriting Techniques for Fine-Grained Access Control. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. ACM, Paris, France, 551–562. doi:10.1145/1007568.1007631

[49] Anadi Shakya, Primal Pappachan, David Maier, Roberto Yus, Sharad Mehrotra, and Johann-Christoph Freytag. 2025. Scalable Enforcement of Fine Grained Access Control Policies in Relational Database Management Systems. arXiv:2511.14629 [cs.DB] doi:10.48550/arXiv.2511.14629

[50] Snowflake Inc. n.d.. Understanding Row Access Policies. https://docs.snowflake.com/en/user-guide/security-row-intro. Accessed 2026-10-26.

[51] Mihail Stoian, Andreas Zimmerer, Skander Krid, Amadou Latyr Ngom, Jialin Ding, Tim Kraska, and Andreas Kipf. 2025. Parachute: Single-Pass Bi-Directional Information Passing. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3299–3311. doi:10.14778/3748191.3748196

[52] Michael Stonebraker and Eugene Wong. 1974. Access Control in a Relational Data Base Management System by Query Modification. In *Proceedings of the 1974 Annual Conference (ACM '74)*. ACM, 180–186. doi:10.1145/800182.810400

[53] K. Tuncay Tekle and Yanhong A. Liu. 2011. More Efficient Datalog Queries: Subsumptive Tabling Beats Magic Sets. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 661–672. doi:10.1145/1989323.1989393

[54] The PostgreSQL Global Development Group. 2023. PostgreSQL 16 Documentation: Row Security Policies. https://www.postgresql.org/docs/16/ddl-rowsecurity.html. Accessed 2026-10-26.

[55] The PostgreSQL Global Development Group. 2023. PostgreSQL 16 Documentation: Table Expressions. https://www.postgresql.org/docs/16/queries-table-expressions.html. Accessed 2026-10-26.

[56] Khai Tran, Sudarshan Vasudevan, Pratham Desai, Alex Gorelik, Mayank Ahuja, Athrey Yadatore Venkateshababu, Mohit Verma, Dichao Hu, Walaa Eldin Moustafa, Vasanth Rajamani, Ankit Gupta, Issac Buenrostro, and Kalinda Raina. 2025. Data Guard: A Fine-grained Purpose-based Access Control System for Large Data Warehouses. arXiv:2502.01998 [cs.DB] doi:10.48550/arXiv.2502.01998

[57] Transaction Processing Performance Council. 2022. TPC Benchmark H Standard Specification, Revision 3.0.1. TPC Standard Specification. https://www.tpc.org/TPC_Documents_Current_Versions/pdf/TPC-H_v3.0.1.pdf Revision dated 28 April 2022.

[58] Nikolaos Tziavelis, Wolfgang Gatterbauer, and Mirek Riedewald. 2021. Beyond Equi-joins: Ranking, Enumeration and Factorization. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2599–2612. doi:10.14778/3476249.3476306

[59] Qichen Wang, Bingnan Chen, Binyang Dai, Ke Yi, Feifei Li, and Liang Lin. 2025. Yannakakis+: Practical Acyclic Query Evaluation with Theoretical Guarantees. *Proceedings of the ACM on Management of Data* 3, 3, Article 235 (2025), 28 pages. doi:10.1145/3725423

[60] Yisu Remy Wang, Max Willsey, and Dan Suciu. 2023. Free Join: Unifying Worst-Case Optimal and Traditional Joins. *Proceedings of the ACM on Management of Data* 1, 2, Article 150 (2023), 23 pages. doi:10.1145/3589295

[61] Yifei Yang, Hangdong Zhao, Xiangyao Yu, and Paraschos Koutris. 2024. Predicate Transfer: Efficient Pre-Filtering on Multi-Join Queries. In *14th Conference on Innovative Data Systems Research (CIDR)*. Chaminade, CA, USA.

[62] Mihalis Yannakakis. 1981. Algorithms for Acyclic Database Schemes. In *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*. Cannes, France, 82–94.

[63] Wen Zhang, Eric Sheng, Michael Chang, Aurojit Panda, Mooly Sagiv, and Scott Shenker. 2022. Blockaid: Data Access Policy Enforcement for Web Applications. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 701–718. https://www.usenix.org/conference/osdi22/presentation/zhang

[64] Hangdong Zhao, Yuanyuan Tian, Rana Alotaibi, Bailu Ding, Nicolas Bruno, Jesús Camacho-Rodríguez, Vassilis Papadimos, Ernesto Cervantes Juárez, César A. Galindo-Legaria, and Carlo Curino. 2026. I Can't Believe It's Not Yannakakis: Pragmatic Bitmap Filters in Microsoft SQL Server. In *Proceedings of the 16th Conference on Innovative Data Systems Research (CIDR)*. https://www.vldb.org/cidrdb/2026/i-cant-believe-its-not-yannakakis-pragmatic-bitmap-filters-in-microsoft-sql-server.html

[65] Jianqiao Zhu, Navneet Potti, Saket Saurabh, and Jignesh M. Patel. 2017. Looking Ahead Makes Query Plans Robust. *Proceedings of the VLDB Endowment* 10, 8 (2017), 889–900. doi:10.14778/3090163.3090167

A Proofs

This appendix gives the proofs that Sections 4 and 5 defer to [1].

Theorem 1 (Correctness). *For every normal form of $\pi[\theta]$, order of children and candidates, and choice of candidate sets satisfying Definition 3, Algorithm 1 returns* $\mathrm{acc}(T,\pi,\theta)$*, and every stored entry $V[([\varepsilon],k)]$ equals $[[\varepsilon']]_b$ for all $\varepsilon' \in [\varepsilon]$ and all b with $\kappa_{\varepsilon'}(b)=k$.*

Proof. We show by induction on the height of n that every call Decide(n,b) returns $[[n]]_b$ and that every entry it records satisfies the stated invariant. Members of $[\varepsilon]$ have equal height, so an expansion never probes its own class, and an entry is written only after an expansion that depends on calls of smaller height alone. A hit therefore reads an entry already shown correct. Atoms are evaluated exactly, and Boolean nodes short-circuit only at determined values. At a quantifier, a hit is correct by the invariant. On a miss, every tuple outside $\mathrm{Cand}(x,\varphi,b)$ violates a mandatory conjunct and falsifies φ, so by induction the expansion records good iff $[[\varepsilon]]_b=1$. By Lemma 1, this outcome holds for every member of $[\varepsilon]$ under every assignment with the same key, and applying Q yields $[[n]]_b$. Calls descend to proper subformulas and candidate sets are finite, so evaluation terminates.

Lemma 2 (Monotone outcomes). *Let ε have an ordered connection with $\omega \in \{>,\geq\}$, and let b and b' agree on the residual key and give $y.B$ the ranks $\rho' \leq \rho$. If $[[\varepsilon]]_b=1$, then $[[\varepsilon]]_{b'}=1$. For $\omega \in \{<,\leq\}$, the same holds for $\rho' \geq \rho$.*

Proof. A witness w under b satisfies γ under b', as ranks preserve order and $\rho' \leq \rho$, and the rest φ' of the body by Lemma 1, as b and b' agree on the residual key and φ' does not read $y.B$. The other case is symmetric.

Theorem 2 (Correctness). *For any traversal plan with co-entered bundles, Algorithm 2 over rank artifacts and the decision store of Section 5.2 returns* $\mathrm{acc}(T,\pi,\theta)$*, independently of evaluation order and of which store entries are skipped or evicted.*

Proof. *Candidates.* Ranks preserve order and nulls have no rank, so atoms and slices evaluated on ranks agree with the value semantics. This also holds for a parameter a absent from the universe, since $d_i < a$ iff $i < \mathrm{lo}(a)$ and $d_i \leq a$ iff $i < \mathrm{hi}(a)$. A slice thus holds exactly the rows that satisfy its connection, a mandatory conjunct, and by (i) one slice serves every member of a bundle. Key returns $\emptyset$ only when that slice is empty, in which case no witness exists.

Reuse. Every answer of Probe other than $\bot$ holds for all assignments with the same key, which extends the invariant of Theorem 1. Equal cores imply equal free-attribute values, as a key determines its row. A key equality reduces ε to φ' on its unique row w, whose row facts are fixed during an evaluation, so equal keys give φ' equal values by Lemma 1. Envelopes are used only when $y.B$ occurs solely in the connection. For $\omega \in \{>,\geq\}$, all ranks up to $\beta_\top$ are good and all ranks from $\beta_\bot$ on are nogood. This holds for the initial values 0 and $m+1$, and by Lemma 2 every Record preserves it. The case $\omega \in \{<,\leq\}$ is symmetric.

Shared walks. We argue by induction on quantifier height, using (ii) and the monotonicity of Kleene logic: a value decided while nested quantifiers are $\bot$ survives every completion. Atoms are exact by the first step and nested outcomes are correct by induction, so a decided t_j equals the value of φ_j on the candidate. Once all required bundles have returned, no quantifier of φ_j is $\bot$, so t_j is decided. A request thus records good only with a witness, and nogood only after lines 7–9 falsified its body on every candidate of a slice that contains all witnesses. Cancellation records nothing, and Propagate combines only closed outcomes through the Boolean nodes of $\pi[\theta]$ (iii). Hence a decided root has the value $[[\pi[\theta]]]_{\{x_T \mapsto r\}}$, and without cancellation every request closes, so the root is decided. Slices are finite, so the walk terminates.

A skipped or evicted entry only makes Probe return $\bot$, which forces an expansion, so the argument holds for every store. It uses no order of candidates, children, or bundles.

B Workloads

This appendix describes every workload of Section 6. Section D.5 gives the shape and the kept rows of each policy.

B.1 Designed Policies

We design the TPC-H and MIMIC policies by role. A role states who may see a row and why, and we write down the policy that this sentence implies. Each tier mixes key joins, ordered comparisons of two attributes, many-to-many connections, and $\neg\exists$ branches in proportions fixed in advance (Table 2). Three policies show how a role turns into a shape. We write a key join in the binder: $\exists o[\ell.orderkey]$ binds o to the order of ℓ.

T14 European trade finance sees items some European supplier could finance.
$\exists s,n[s.nationkey]: s.acctbal \geq \ell.extendedprice \wedge n.regionkey=3$

T13 Domestic trade compliance sees urgent items whose customer and supplier share a nation.
$\exists o[\ell.orderkey],s[\ell.suppkey],c[o.custkey]: s.nationkey=c.nationkey \wedge \exists o[\ell.orderkey]: o.orderpriority=$'1-URGENT'

T28 Trade desk sees domestic items or items a European supplier could finance, except open urgent orders.
$(\exists o[\ell.orderkey],s[\ell.suppkey],c[o.custkey]: s.nationkey=c.nationkey \vee \exists s,n[s.nationkey],r[n.regionkey]: s.acctbal \geq \ell.extendedprice \wedge r.name=$ 'EUROPE') $\wedge \neg\exists o[\ell.orderkey]: o.orderpriority=$'1-URGENT' $\wedge o.orderstatus=$'O'

T14 reaches the lineitem only through an ordered comparison, which no key serves. One envelope (Section 5.2), filled from the European suppliers, therefore decides every lineitem. T13 closes a cycle through the nation, the diamond of Section 3, and many lineitems share each pair of nations. T28 joins the domestic branch of T13 and the branch of T14 by $\vee$ and adds a $\neg\exists$, so that a positive and a negative branch enter through the same order. The roles of all designed policies follow.

TPC-H

Simple
- T01 Air freight desk sees air shipments.
- T02 Claims sees returned items received after their commit date.
- T03 Pricing audit sees heavily discounted bulk lines.
- T04 Warehouse sees the items of urgent orders.
- T05 Fulfilment sees items whose supplier keeps at least 3000 units of the part in stock.
- T06 Finance sees items not yet returned, of orders that are no longer open.
- T07 Trade finance sees small lines, or lines some German supplier could finance from its balance.
- T08 Truck planning sees truck shipments sent on a day a large urgent order was placed.
- T09 Dock planning sees items delivered in person, or received on a day with no large urgent order.
- T10 Brand manager sees items of its brand in medium boxes.

Intermediate

T11 French country manager sees the items of French customers.
T12 Household desk sees items of household customers, except low priority orders.
T13 Domestic trade compliance sees urgent items whose customer and supplier share a nation.
T14 European trade finance sees items some European supplier could finance.
T15 Logistics sees items shipped on a heavy urgent day or supplied from Japan.
T16 Dock planning sees items not received on a congested day and not bought by a customer in debt.
T17 The automobile desk hides items of automobile customers unless the order is urgent.
T18 Collections hides the open orders of customers in debt.
T19 The Asia desk sees items of Asian customers whose orders are not yet fulfilled.
T20 Credit sees items a German supplier could finance, or whose customer is not in debt.

Complex

T21 Asian regional manager sees items bought from a supplier in the customer's own nation (after TPC-H Q5).
T22 American market analysts see items of a product type bought by American customers, unless the customer is in debt (after TPC-H Q8).
T23 Fulfilment sees domestic items whose supplier stocks enough of the part for the line, unless the customer is in debt.
T24 Profit analysts hide the open order items supplied from Europe and ordered from 1995 on (after TPC-H Q9).
T25 Saudi supplier performance sees late items of fulfilled orders, unless the Saudi supplier is low on a large part.
T26 EU data protection sees items that involve a European customer or a European supplier.
T27 Asian credit sees items unless bought from a richer supplier in the customer's own nation, and unless the customer is in debt.
T28 Trade desk sees domestic items or items a European supplier could finance, except open urgent orders.
T29 Africa desk sees items of African customers, and any item not shipped on a congested day.
T30 American brand team sees urgent domestic items of its brand from well stocked suppliers.

MIMIC

Simple

M01 Alarm desk sees charted values that raised a warning.
M02 Hypertension watch sees pressure readings of 180 mmHg or more.
M03 Respiratory and cardiac monitors see rate readings without a warning.
M04 Vital signs dashboard sees routine vital signs and every value that raised a warning.
M05 Clinical review sees values above the item's normal range.
M06 Quality review sees charts except those of admissions that ended in death, but always sees values that raised a warning.
M07 A caregiver sees their charts in stays where they gave norepinephrine.
M08 Ventilation team sees respiratory rate readings recorded during invasive ventilation.
M09 Bedside review sees blood pressure readings not taken at the moment a lab was drawn.
M10 Sedation-free review sees charts recorded while the patient was not under propofol.

Intermediate

M11 Medicaid audit sees MICU charts of Medicaid admissions.
M12 MICU respiratory team sees respiratory charts of MICU stays.
M13 A bereavement rule hides the charts of admissions of patients 75 or older that ended in death.
M14 Unit staff see charts recorded while the patient was in the first unit of the stay.
M15 Critical care sees charts of stays on norepinephrine or on invasive ventilation.
M16 The sepsis team sees charts of admissions with a sepsis diagnosis, unless the admission has a do not resuscitate code.
M17 The ventilation team sees charts recorded during invasive ventilation, except stays on dialysis (CRRT).
M18 A caregiver sees their charts in stays where they gave an antibiotic, unless the stay received propofol.
M19 Research export hides charts recorded at the moment a blood gas was drawn and charts of admissions that ended in death.
M20 Charts are visible unless the admission has a mental health diagnosis (ICD-10 chapter F or ICD-9 290 to 319), then only to caregivers who gave an input in that stay.

Complex

M21 Kidney research sees vital signs of ICU patients 50 or older with acute kidney failure.
M22 A clinician sees charts of ventilated ICU admissions of patients 40 or older in which they placed an order, unless the admission has a do not resuscitate code.
M23 Hypertension research sees charts of medicine admissions with hypertension, a procedure and Foley output, where the patient moved to the last ICU unit.
M24 A clinician sees charts of severe urgent admissions in which they placed an order, or charts of patients 60 or older with a resistant culture recorded in the first unit of the stay.
M25 Electrolyte review sees all charts of patients under 65, and charts of patients 65 or older only when recorded in the ICU at the moment an abnormal blood potassium was drawn.
M26 Critical care research sees charts of severe medicine admissions or charts recorded during ventilation, except admissions with a depression diagnosis.
M27 General ward research hides charts of patients 50 or older with acute kidney failure and charts of admissions in the most severe APR DRG.
M28 Geriatric ICU sees charts of patients 70 or older recorded under propofol, except during a cardiac surgery service stay with a procedure.
M29 A dosing blind study hides charts of patients 40 or older recorded after an administered potassium chloride dose.
M30 A clinician sees charts of their admissions, or first unit charts unless the admission has a do not resuscitate code, or charts recorded when an abnormal STAT chemistry was drawn.

The SQL of each of these policies is in [1], together with the JOB policies and the inputs of the multiplicity, parametricity, and width workloads.

B.2 JOB Policies

A JOB query yields a template on `title` as Section 6.1 describes. Cutting a query keeps whole paths from `title` to the filtered relations, so that a cut query is again a valid policy. The tiers thus hold the same families at different sizes, which lets us compare a policy with its own cut versions. In 25 of the 30 policies, a branch enters from the primary key of `title`, so its decisions belong to one target row and leave little for the store to reuse (Table 2, column own key).

B.3 Multiplicity

Both policies run on copies of `customer` and `supplier` with one added column per k, each with an index on the group and the balance. Group keeps a customer when its group lies in a fixed fifth of the groups, and needs no comparison with the customer, whereas Balance reads the balance of the customer. The constant θ_k of Balance must rise with k to keep 20% of the customers (Table 3). A customer needs a supplier of its own group with a balance between its own and θ_k, so the fewer suppliers a group has, the higher θ_k must reach. The exact values are $\theta_k = 1{,}201.92$ at $k = 5$ and 3,047.35 at $k = 25{,}000$, and Balance keeps between 299,990 and 300,014 customers. At $k = 25{,}000$, a group has only 4 suppliers, and 21,046 of the 25,000 groups still hold one under θ_k. Section D.2.1 shows what the fan-out does to each method.

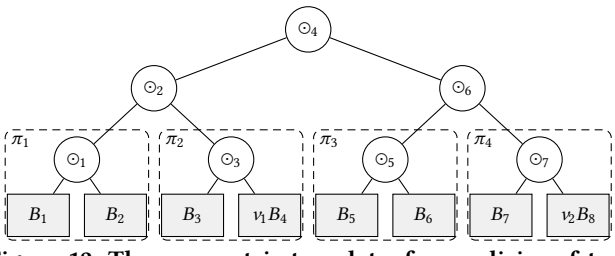
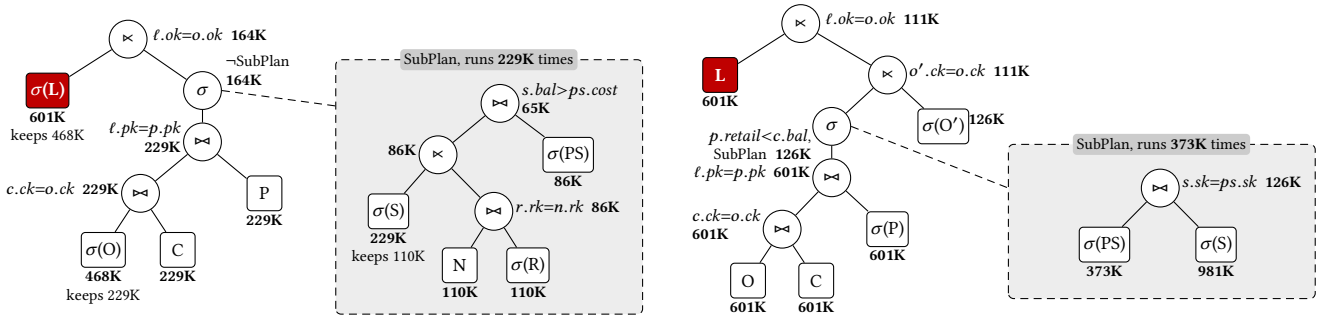


Figure 12: The parametric template: four policies of two branches each.



(a) B_1 **under P1, with a nested scope.** **(b)** B_3 **under P1, with a scope beside the offer.**

Figure 13: PostgreSQL's plans for the correlated form of two branches at SF 0.1, measured by `EXPLAIN ANALYZE`. All joins are nested loops. Bold numbers give the rows an operator returns and, at a leaf, the rows scanned or the index probes. A join shows the predicate its inner index scan applies. A dashed box is a SubPlan, which the σ linked to it runs per row.

B.4 Parametric Template

Template. The parametricity experiments use one template on `lineitem`, which Figure 12 shows. It consists of four parametric policies of two branches each, joined by connectives that the binding sets:

$$\pi_1 = B_1 \odot_1 B_2, \quad \pi_2 = B_3 \odot_3 \nu_1 B_4, \quad \pi_3 = B_5 \odot_5 B_6, \quad \pi_4 = B_7 \odot_7 \nu_2 B_8,$$
$$\pi = (\pi_1 \odot_2 \pi_2) \odot_4 (\pi_3 \odot_6 \pi_4),$$

where each $\odot_i$ is $\wedge$ or $\vee$ and each ν_i is a negation or empty.

Branches. Every branch has 16 predicates on three levels of quantifiers, so the template has 128. The outer $\exists$ binds the order, customer, and part of the line by their keys. Below it, an *offer* asks whether some supplier of the part qualifies, either the supplier of the line or another one, in the nation of the customer or elsewhere, with a balance above or below the cost. The polarity Q of the offer decides whether such an offer is required or forbidden. A *scope* then compares the line with rows related to it. In B_1, B_2, and B_5, the scope sits inside the offer and looks at the nation and region of the supplier, other offers of the part of the line, or customers of the same nation and segment. In the other five, it sits beside the offer and looks at other orders of the customer, the supplier of the line, other lines of the part, parts of the same brand and container, or other lines of the order. Alone, a branch keeps 15% to 58% of the lineitems at SF 0.1. Figure 13 shows what PostgreSQL makes of the two shapes when it receives the policy as correlated subqueries. It scans lineitem, unnests the outer $\exists$ into a semijoin, and keeps the offer as a SubPlan that runs for each line that survives the filters: 229K times for B_1 and 373K times for B_3, on a table of 601K lines. This is the repeated witness search of Section 3, and the joins below the semijoin read the order, customer, and part of each line one by one. Which relation a SubPlan enters first depends on the binding: in B_1, ω_1 is $=$, so the plan enters through the supplier of the line, whereas in B_3, ω_8 is $\neq$, and it enters through all offers of the part. All eight outer quantifiers enter through equal connections, and so do the eight offers, so that one walk can serve them as a bundle (Section 5.3). The scopes differ on purpose: the scope over other lines of a part reaches about 30 rows per line at SF 10, whereas the scope over a nation reaches one.

Bindings. A binding reads the template for one role. The role fixes the Boolean form, whether each offer is required or forbidden, and which comparisons apply, and the range constants of a branch then come from a fixed list. Table 4 gives the eight roles and the form each one sets. Inside the branches, a binding also sets 7 polarities, 31 operators, and 14 constants (Tables 6 and 7), so that 44 of the 128 predicates depend on it.

Table 4: The eight bindings: role, Boolean form over the branches, and lineitems kept.

binding	role	form over branches	kept
P1	Procurement: a line needs a funded alternative offer	$((1\vee2)\wedge(3\vee\neg4))\vee((5\vee6)\wedge(7\vee8))$	52%
P2	Credit: underfunded suppliers are visible only with buyer cover	$((1\wedge2)\vee(3\wedge\neg4))\vee((5\wedge6)\vee(7\wedge8))$	53%
P3	Compliance: domestic sourcing required unless no domestic offer exists	$((1\vee2)\vee(3\wedge4))\wedge((5\vee6)\vee(7\vee\neg8))$	46%
P4	Logistics: late or short lines with a backup offer	$((1\vee2)\wedge(3\vee4))\vee((5\wedge6)\vee(7\wedge\neg8))$	67%
P5	Audit: a line is reviewable when a peer record contradicts it	$((1\wedge2)\vee(3\vee4))\wedge((5\vee6)\vee(7\wedge8))$	34%
P6	Returns: returned or disputed lines with a cheaper source	$((1\vee2)\vee(3\wedge\neg4))\vee((5\wedge6)\wedge(7\vee8))$	60%
P7	Regional: Europe buyers against offers from outside the region	$((1\wedge2)\vee(3\wedge4))\vee((5\vee6)\wedge(7\vee\neg8))$	47%
P8	Finance: exposure lines where supplier and buyer balances disagree	$((1\vee2)\vee(3\vee\neg4))\wedge((5\wedge6)\vee(7\vee8))$	50%

Eight effective policies, one template. One binding cannot show parametricity, since a plan tuned for it is simply a good plan. The eight bindings therefore vary every kind of parameter at once: each connective, negation, polarity, operator, and constant takes at least two values among them. Their results stay comparable, between 34% and 67% of the lineitems, so that neither a selective filter nor an empty result decides the time. To a plan, they are eight unrelated queries. Each differs from P1 in 14 to 22 predicates and leaves at most 2 of its 8 branches as they are (Table 5), and only 16 of the 64 branches repeat exactly across all bindings. A new operator can invalidate a `MIN` or `MAX` rewrite, and a new polarity turns a semijoin into an antijoin. To the walk, they are one policy. The other 84 predicates never change, all key connections stay, and every binding compares the same attributes of the same relations, so the rank artifacts and the positions of the decisions are the same. A binding only decides how far each walk goes.

How many policies. The eight bindings are a small sample. With only the values they already use, the template has about 2×10^{19} instances, 2^{47} choices of form, polarity, and operator times 165,888 combinations of constants, and about 10^{34} if each operator may be

Table 5: How far each binding is from P1.

binding	connectives	polarities	operators	constants	predicates	same branches
P2	6	6	10	8	18	1
P3	6	4	10	6	16	2
P4	5	4	14	5	19	1
P5	6	6	14	8	22	1
P6	3	4	7	7	14	2
P7	5	4	9	5	14	1
P8	4	4	10	6	16	2

any of the six in Ω. No system can prepare a plan or a rewrite for each, so the artifacts must not depend on the binding.

Table 6: Polarities Q and operators ω that each binding sets.

variable	P1	P2	P3	P4	P5	P6	P7	P8
Q_1	¬∃	∃	∃	¬∃	¬∃	∃	¬∃	¬∃
Q_2	∃	¬∃	¬∃	¬∃	¬∃	∃	¬∃	¬∃
Q_3	∃	¬∃	∃	¬∃	¬∃	¬∃	¬∃	∃
Q_4	∃	¬∃	¬∃	∃	¬∃	¬∃	∃	¬∃
Q_5	¬∃	∃	¬∃	¬∃	∃	¬∃	¬∃	¬∃
Q_6	¬∃	∃	¬∃	∃	∃	∃	∃	∃
Q_7	¬∃	¬∃	∃	∃	∃	¬∃	∃	∃
ω_1	=	=	≠	≠	=	≠	≠	=
ω_2	>	>	<	<	>	>	>	<
ω_3	≠	≠	=	≠	=	≠	≠	≠
ω_4	>	>	>	<	>	>	>	<
ω_5	<	<	<	<	>	<	<	<
ω_6	<	<	>	<	<	<	<	<
ω_7	≠	≠	≠	≠	≠	=	≠	≠
ω_8	≠	≠	=	≠	=	≠	≠	=
ω_9	≠	=	≠	=	≠	≠	=	≠
ω_{10}	<	>	>	>	<	<	>	>
ω_{11}	≠	≠	≠	≠	≠	=	≠	≠
ω_{12}	>	>	>	≤	>	>	>	>
ω_{13}	=	=	≠	=	≠	=	=	=
ω_{14}	>	>	<	>	<	>	>	>
ω_{15}	>	>	>	>	>	>	>	<
ω_{16}	>	>	>	>	>	>	>	<
ω_{17}	<	>	<	>	<	<	<	<
ω_{18}	<	<	<	<	>	<	<	<
ω_{19}	>	>	>	>	<	>	>	>
ω_{20}	≠	≠	≠	≠	=	≠	=	≠
ω_{21}	=	≠	=	≠	≠	=	≠	=
ω_{22}	>	<	>	<	>	>	<	>
ω_{23}	<	<	<	<	>	<	<	<
ω_{24}	<	<	>	<	<	>	<	>
ω_{25}	<	<	<	<	<	<	>	<
ω_{26}	≠	=	≠	=	=	=	=	=
ω_{27}	<	>	<	>	>	>	>	>
ω_{28}	<	>	<	>	>	<	<	<
ω_{29}	<	>	<	>	>	<	<	<
ω_{30}	≠	≠	=	≠	≠	=	≠	≠
ω_{31}	>	<	>	<	>	>	>	<

Table 7: Constants that each binding sets.

variable	P1	P2	P3	P4	P5	P6	P7	P8
$1	180000	320000	260000	260000	320000	320000	180000	260000
$2	EUROPE	EUROPE	EUROPE	EUROPE	EUROPE	EUROPE	ASIA	EUROPE
$3	0	0	0	0	0	F	0	0
$4	4000	7000	5000	4000	6000	4000	7000	4000
$5	1993-01-01	1993-01-01	1994-01-01	1993-01-01	1993-01-01	1993-01-01	1993-01-01	1993-01-01
$6	1998-01-01	1998-01-01	1996-01-01	1998-01-01	1998-01-01	1997-01-01	1998-01-01	1998-01-01
$7	A	A	A	A	A	N	A	A
$8	4000	5000	5000	4000	5000	5000	5000	5000
$9	6000	5500	6000	6000	6000	5300	6000	5500
$10	0.01	0.01	0.01	0.01	0.02	0.01	0.01	0.01
$11	0.07	0.05	0.07	0.05	0.05	0.07	0.07	0.07
$12	1200	1100	1200	1100	1100	1300	1100	1100
$13	1450	1600	1450	1600	1600	1450	1600	1600
$14	0.05	0.07	0.07	0.07	0.07	0.05	0.05	0.07

B.5 Width

The width experiments use branches of the same shape. W_k joins k branches by the form in Table 8. Each step adds two branches, about 6 quantifiers, and 32 predicates, while every width keeps about 60% of the lineitems, so that a change in time comes from the width and not from the result.

Table 8: Width policies. Quant.: quantifiers. Rel.: relation occurrences. Join, ineq., const.: predicates by kind.

policy	form over branches	quant.	¬∃	rel.	join	ineq.	const.	kept
W_2	1∨2	6	1	14	13	13	6	58%
W_4	((1∨2)∧¬3)∨4	12	4	26	26	25	13	58%
W_6	((1∨2)∧(3∨¬4))∨(5∧6)	18	5	38	39	38	19	57%
W_8	((1∨2)∧(3∨¬4))∨((5∨6)∧(7∨8))	24	6	50	54	50	24	59%
W_{10}	(((1∨2)∧(3∨¬4))∨((5∨6)∧(7∨8)))∨(9∧¬10)	30	7	62	67	63	30	58%

B.6 Scale

The scale experiments run the 30 TPC-H policies unchanged on the TPC-H databases of scale factors 0.1, 1, 10, and 100, with artifacts built for each.

C Extra Optimizations

The engine implements the walk of Section 4.3 and the reuse of Section 5.2 as the paper describes them, and adds seven optimizations on top. Each one is chosen when the policy is compiled, from a property of the policy and its artifacts, and none changes which rows a policy allows. Where an optimization cannot answer a target row, the row is decided by the walk of Algorithm 1, which therefore remains the reference for every result. Table 9 gives the number of policies of each dataset in which the mechanisms of Section 5.2 and the extra optimizations apply.

Table 9: Number of the 30 policies of each dataset in which a mechanism applies.

mechanism	MIMIC	TPC-H	JOB
Section 5.2			
core keys	27	27	17
key-equality keys with row facts	6	19	0
monotone outcomes	10	5	0
extra optimizations			
compiled target loop	30	30	30
in blocks of 1,024 rows	30	28	11
kept chains	11	22	11
composed boundaries	9	5	0
multi-equality entry	10	8	0
component factoring	5	4	20
target rank cache	4	2	0
branch reordering	4	7	9

Core keys apply to every policy that stores a decision. Shared walks (Section 5.3) are counted on the nested policies of Appendix D.2.4, all 13 of which share nodes across their branches, from 8 in W2 to 20 in W10.

Compiled target loop. The walk forms keys, probes the store, and expands the misses for every target row. For the root of the decision graph, the engine compiles this into a loop that, for each target row, loads the entering rank of each branch from the forward maps, reads the cell that the branch keeps for that rank, and combines the outcomes by the Boolean form of the root. An empty cell sends its branch to the walk, which fills the cell, so the loop decides every row as the walk would. In 69 of the 90 policies, the loop also runs over blocks of 1,024 target rows and fills 16 words of the bitmap at a time. A simple TPC-H policy thus costs a few nanoseconds per lineitem.

Kept chains. Row facts (Section 5.2) let a key equality form the key of its quantifier without reading again the row it reaches. A kept chain extends them along a path of key equalities, for example from a lineitem to its order and on to the customer of the order. The ranks that the rest of the branch compares at the end of the

path are kept under the rank where the path starts, so a later target row with the same rank reads them in one step. In 42 of the 44 policies with a kept chain, the kept ranks decide at least one branch completely.

Composed boundaries. A monotone outcome (Lemma 2) belongs to one ordered connection. When it lies at the end of a path of key equalities, the engine keeps its boundary under the rank where the path starts. The boundary is filled on the first arrival at that rank, from complete witnesses only, and a target row then compares its own value with it instead of walking the path. In T23, filling these boundaries takes a third of the time, which Figure 18 counts as evaluation.

Multi-equality entry. A quantifier entered through several equalities, such as a partsupp row reached through both its part and its supplier, may use the slice of any of them. The engine iterates one slice and tests the other equalities on its rows. Where the equalities together form a key, it keeps the outcome of the one row they reach under its position in that slice, so a later arrival at the same row reuses it. In seven MIMIC policies, the quantifier is instead entered through two equalities at once.

Component factoring. When the conjuncts of a scope fall into groups that share only variables bound outside the scope, the engine decides each group on its own and keeps its outcome under its own key, so one group is reused even when another changes. This applies most in JOB, whose scopes join many relations.

Target rank cache. When the whole policy depends on the target only through the rank of one column, its outcome is kept under that rank, and every later target row with the same rank is decided by one lookup.

Branch reordering. The walk orders the children of a node by the lengths of their slices (Section 4.3). At the root, the engine also moves a branch ahead when its expected work, computed from the sizes in the artifacts, is at most half of that of the branch before it, since a branch that decides the root early spares the others. The expected work is an estimate, so a wrong estimate costs time but never changes a decision.

D Detailed Results

This appendix extends Section 6. Its subsections follow those of Section 6, and each gives the numbers behind the figures of its counterpart together with the plans and counters that explain them. Times come from the runs of Section 6, plans from `EXPLAIN ANALYZE` on the same databases, and reads and decisions from the counters of the walk.

D.1 Comparison with Existing Enforcement

Section 6.2 compares Memento with six methods on 90 policies (Figure 7). Table 10 aggregates this comparison by tier. The advantage of Memento over the PostgreSQL methods grows with the tier, whereas $\text{Bespoke}_{\text{Duck}}$ stays close on the simple and intermediate tiers.

Where Memento loses. Table 10 compares Memento with each method separately. Against the fastest method of each policy, Memento is fastest on 48 of the 90 policies. Whether it wins depends mostly on whether the walk can reuse its decisions. The witness tuples the walk reads per target row show this. When many target rows share a decision, the walk reads far fewer than one witness per row. When every row needs its own decision, it reads at least one. Table 11 groups the policies in this way.

Table 10: Policies of 30 on which Memento is faster, and in parentheses the median speedup.

tier	PG-RLS	DBMS-O	$\text{Verbatim}_{\text{PG}}$	$\text{Bespoke}_{\text{PG}}$	$\text{Verbatim}_{\text{Duck}}$	$\text{Bespoke}_{\text{Duck}}$
simple	27 (7.1×)	28 (12.5×)	24 (3.2×)	25 (3.4×)	20 (1.6×)	16 (1.2×)
intermediate	30 (26.4×)	28 (8.4×)	25 (6.7×)	26 (6.3×)	21 (2.0×)	15 (1.0×)
complex	30 (58.3×)	29 (31.0×)	29 (32.3×)	29 (13.2×)	25 (3.8×)	21 (2.2×)

Table 11: Memento against the fastest method of each policy, medians per group.

	policies	Memento fastest	reads per row	ns per row
filter only	6	3	0.00	12
decisions not reused	16	1	2.43	409
decisions reused	68	44	0.06	37

Filter only. Six policies only filter the target. There is nothing to decide, every method scans the same columns, and a vectorized scan is as fast as the walk, so Memento is fastest on 3 of them.

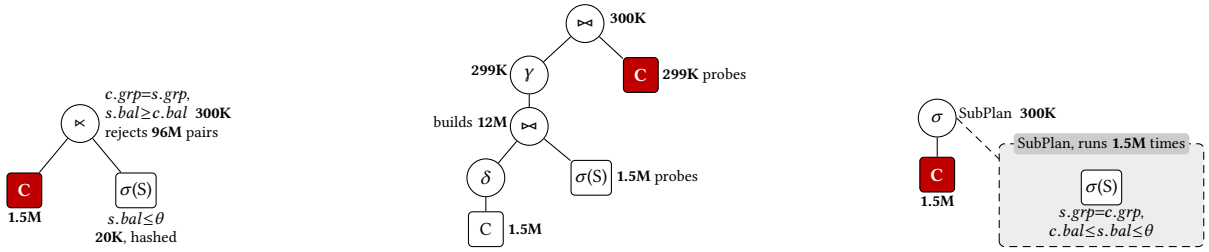
Decisions not reused. In 16 policies, the walk reads at least one witness for every target row. All but T05 are JOB policies that enter from the key of `title` (Section B.2), so a decision belongs to a single row and the store has nothing to reuse. The walk then looks up the witnesses of each row separately, at a median of 409 ns per target, and a join plan is faster on 15 of the 16. T05 is the only designed policy among them. Its walk reads 3.5 rows of `partsupp` per lineitem, and it is 10× slower than $\text{Bespoke}_{\text{Duck}}$.

Decisions reused. In the other 68 policies, the walk reads a median of 0.06 witnesses per row, and Memento is fastest on 44. Of its 24 losses there, 22 go to a DuckDB plan. These are short queries: on them, the fastest method takes a median of 32 ms and Memento 210 ms. T10 shows a case that the walk cannot avoid. It keeps 0.1% of the lineitems. $\text{Bespoke}_{\text{PG}}$ starts from the 2,044 parts that pass and fetches their 61K lineitems through an index, whereas the walk starts from the target and visits all 60M lineitems at 37 ns each, 2.2 s in total.

Plan shapes. $\text{Verbatim}_{\text{PG}}$ unnests 28 of the 46 TPC-H branches into joins, runs 13 as subqueries that are computed once and hashed, and runs 5 once per lineitem, in T07, T20, T24, and T28 (Table 12). PG-RLS unnests none. PostgreSQL turns a subquery into a join only when it occurs in the query itself, and it adds a row-level security policy as a separate qualification, so every branch stays a subquery: 31 hashed and 15 run per row, in 12 of the 30 policies.

Both methods time out on the same 4 policies, T07, T14, T20, and T28. Under $\text{Verbatim}_{\text{PG}}$, three of them run a subquery per row. In T07, every lineitem with more than 5 units scans the German suppliers until one can finance it. T14 is unnested there, but its only join predicate, $s.acctbal \geq \ell.extendedprice$, is an ordered comparison that no hash serves, so PostgreSQL joins the lineitems with the European suppliers in a nested loop. The walk keeps one envelope for this comparison (Section 5.2) and decides each lineitem with one comparison of ranks, in 0.56 s for T14 and 0.71 s for T07. $\text{Bespoke}_{\text{PG}}$ unnests every branch and finishes all 30, at the cost the next table shows.

Tuples. Table 13 counts the base tuples each method reads and the intermediate tuples it builds. Memento reads every target row and the witnesses it needs, a median of 65M to 80M tuples per



(a) Verbatim$_{PG}$: 9.1 s. **(b) Bespoke$_{PG}$: 6.5 s.** **(c) PG-RLS: 2.9 s.**

Figure 14: PostgreSQL's plans for Balance at $k = 250$, measured by `EXPLAIN ANALYZE`. Bold numbers give the rows an operator returns and, at a leaf, the rows scanned or the index probes. δ removes duplicates and γ groups. Memento takes 28 ms on the same policy.

Table 12: How PostgreSQL plans the branches of the TPC-H policies: as a join, as a subquery run once and hashed, or as a subquery run per row.

		Verbatim$_{PG}$			PG-RLS		
tier	branches	join	hashed	per row	join	hashed	per row
simple	7	5	1	1	0	6	1
intermediate	18	10	7	1	0	15	3
complex	21	13	5	3	0	10	11
all	46	28	13	5	0	31	15

policy, almost the same in every tier. A plan can read fewer, as Verbatim$_{PG}$ does for T02, which reads 15M of the 60M lineitems through an index, but it pays in tuples it builds. Bespoke$_{PG}$ builds more intermediate tuples than `lineitem` has rows in 20 of the 30 policies, up to 800M for T24. Memento builds none.

Table 13: Tuples read and built on TPC-H, median per tier. A superscript counts plans that did not finish and are left out.

	Verbatim$_{PG}$		Bespoke$_{PG}$		Memento	
tier	read	built	read	built	read	built
simple	38M	61K[1]	49M	25M	65M	0
intermediate	70M	16M[2]	69M	158M	78M	0
complex	233M	120M[1]	66M	175M[1]	80M	0

D.2 Controlled Experiments

Section 6.3 varies one property of a policy at a time. This subsection gives the numbers behind each of its figures and explains them with the plans of the baselines and the counters of the walk. It also varies two properties that the section leaves fixed, the share of rows a policy keeps and the depth of its chain.

Table 14: Multiplicity: warm time in seconds.

policy	k	PG-RLS	Verbatim$_{PG}$	Bespoke$_{PG}$	Verbatim$_{Duck}$	Bespoke$_{Duck}$	Memento
Group	5	0.23	0.05	0.043	0.022	0.009	0.008
	25	0.523	0.055	0.213	0.022	0.013	0.008
	250	0.528	0.056	0.214	0.023	0.013	0.008
	2,500	0.54	0.065	0.215	0.025	0.013	0.009
	25,000	0.612	0.19	0.22	0.03	0.014	0.014
Balance	5	5.10	433	506	42.5	45.7	0.028
	25	4.77	86.8	114	10.0	10.7	0.028
	250	2.89	9.14	6.50	1.26	1.39	0.028
	2,500	2.46	1.31	2.74	0.347	0.332	0.028
	25,000	2.07	0.547	1.61	0.257	0.235	0.036

D.2.1 Multiplicity. Table 14 gives the times of Figure 8. Plans and counters explain them.

Table 15: Group: the work of Memento.

k	reads	decisions	ns per customer
5	80,001	5	2.1
25	80,005	25	2.1
250	80,050	250	2.3
2,500	80,500	2,500	2.9
25,000	85,000	25,000	6.4

Table 16: Balance: the work of the two PostgreSQL forms.

	Verbatim$_{PG}$		PG-RLS	
k	rejected pairs	ns each	index probes	μs each
5	4.8B	91	1.5M	3.4
25	952M	91	1.5M	3.2
250	96M	95	1.5M	1.9
2,500	11M	124	1.5M	1.6
25,000	1.6M	336	1.5M	1.4

Group. The walk asks one question per group, and the answer costs little either way (Table 15). A group outside the fixed fifth has no witness, so the walk reads its whole slice once and stores nogood. A group inside stops at its first witness. Four fifths of the suppliers are thus read once, and each passing group adds one read, which gives exactly $80{,}000 + k/5$ reads. Multiplicity changes only how many decisions the walk stores. Deciding a customer takes 2.1 ns at $k = 5$ and 6.4 ns at $k = 25{,}000$, as the store grows from 5 to 25,000 decisions. A plan reads fewer suppliers, since it filters them to the fixed fifth first, and then fetches the customers of those groups.

Balance. Three plans treat the same comparison in three ways (Figure 14). Verbatim$_{PG}$ unnests the policy into a hash semijoin on the group and tests the balance as a join filter. A customer that fails compares with every supplier of its group under θ_k, so time follows the pairs it rejects (Table 16), at about 100 ns per rejected pair until the scan of the customers dominates at large k. These are far fewer than the 30B pairs of the join on the group alone (Table 3). At $k = 5$, the index keeps only the 19,820 suppliers under θ_k, about 3,964 per group, and a customer that passes stops at its first witness, so the 1.2M customers that fail test about 3,964 suppliers each, 4.8B pairs in total. Bespoke$_{PG}$ first builds every qualifying pair, 12M at $k = 250$, and removes duplicates, which is the witness blowup of Section 3. DuckDB runs the same two forms on a vectorized engine, and its time also falls with the fan-out. PG-RLS is the surprise. It keeps the subquery, and the index on group and balance answers each customer with one range probe, whatever the size of the group:

1.5M probes at every k, and no pair to reject (Table 16). Its plan is the simplest, yet it is 85× faster than Verbatim$_{PG}$ at $k=5$: here, unnesting gives up the one structure that decides the comparison. The walk asks the same question, but once per group instead of once per customer. It reads the 100,000 suppliers once, keeps one envelope per group (Section 5.2), and answers the other customers by comparing two ranks, at about 15 ns each. This is why it is flat in k and 182× faster than PG-RLS at $k=5$.

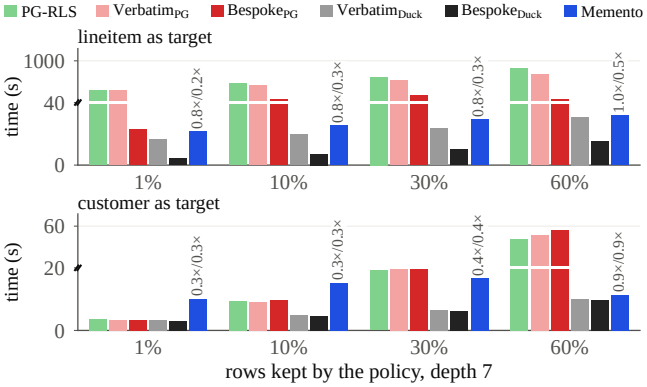


Figure 15: Impact of selectivity at SF 10. Numbers on blue bars show speedup against best Verbatim/Bespoke baselines.

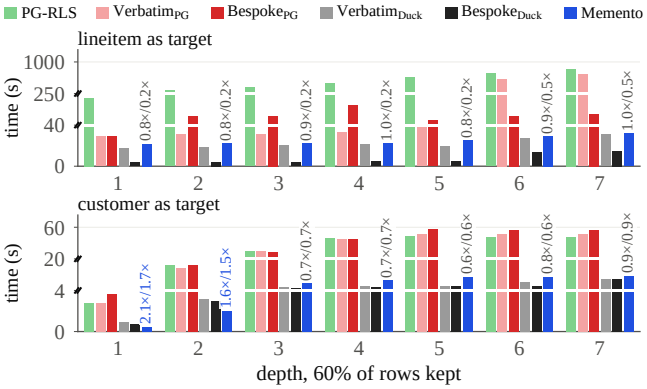


Figure 16: Impact of depth at SF 10. Numbers on blue bars show speedup against best Verbatim/Bespoke baselines.

D.2.2 Selectivity. The policies of this subsection and the next are single chains of ∃ steps from a target. Each step enters its relation by key with a filter, and most steps also compare a value with one already bound. From lineitem the chain reaches partsupp, supplier, nation, region, part, orders, and customer, in this order. From customer it reaches orders, lineitem, partsupp, part, supplier, nation, and region. Constants set the share of target rows the chain keeps. Figure 15 keeps all seven steps and varies this share from 1% to 60%. Table 17 gives the times and the work of Memento per target row. Depth 7 with 60% kept is the same policy in both experiments, so the table lists it in both.

Lineitem. Every method slows as more lineitems pass. From 1% to 60% kept, PG-RLS grows from 323 s to 830 s, Bespoke$_{Duck}$ from 4.3 s to 14.9 s, and Memento from 21.6 s to 31.8 s. Memento is faster than the three PostgreSQL methods at every share, but both DuckDB forms stay ahead. Across the shares, however, Memento slows by

Table 17: Selectivity and depth: warm time in seconds, and reads and node arrivals of Memento per target row.

		PG-RLS	Verbatim$_{PG}$	Bespoke$_{PG}$	Verbatim$_{Duck}$	Bespoke$_{Duck}$	Memento	reads	arrivals
lineitem as target									
kept	1%	323	335	22.9	16.6	4.27	21.6	4.05	1.12
	10%	475	444	112	19.7	7.06	25.4	4.16	1.37
	30%	633	564	207	23.6	10.2	29.4	4.23	1.70
	60%	830	695	112	30.7	14.9	31.8	4.28	2.08
depth	1	221	29.1	29.2	17.9	3.47	21.8	4.00	1.00
	2	326	30.9	97.0	18.0	3.95	22.2	4.00	1.12
	3	399	31.2	97.8	20.7	3.89	22.2	4.00	1.12
	4	493	32.6	172	21.2	4.48	22.3	4.00	1.13
	5	639	38.8	72.6	19.0	4.60	25.0	4.03	1.21
	6	724	576	95.7	27.5	13.2	29.3	4.26	1.83
	7	830	695	112	30.7	14.9	31.8	4.28	2.08
customer as target									
kept	1%	3.60	3.25	3.28	3.35	3.14	10.1	29.9	7.62
	10%	9.44	9.13	9.64	4.91	4.69	15.3	48.8	12.8
	30%	19.2	20.0	21.2	6.58	6.32	16.7	56.8	15.4
	60%	47.4	50.9	56.0	9.94	9.72	11.3	36.9	11.6
depth	1	2.76	2.76	3.62	0.96	0.78	0.46	3.39	1.00
	2	17.0	15.5	16.6	3.15	2.95	2.01	4.76	2.00
	3	29.2	29.4	28.7	5.57	5.40	7.49	23.2	6.61
	4	45.6	44.7	44.4	6.14	5.95	8.99	27.2	8.01
	5	48.5	51.5	57.1	6.16	6.00	10.9	35.0	10.8
	6	47.9	51.3	56.1	8.33	6.10	10.9	35.2	11.0
	7	47.4	50.9	56.0	9.94	9.72	11.3	36.9	11.6

1.5×, less than any other method, while Bespoke$_{Duck}$ slows by 3.5× and Bespoke$_{PG}$ by 4.9×, so the lead of Bespoke$_{Duck}$ shrinks from 5.1× with 1% kept to 2.1× with 60%. The counters show why. The walk decides every one of the 60M lineitems and reads about four rows per lineitem at every share, from 4.05 to 4.28. What grows with the share is how far a walk goes: a lineitem that passes the early steps arrives at later nodes, from 1.12 node arrivals per lineitem at 1% to 2.08 at 60%. A plan applies the filters first and joins only the rows that pass them, so with few rows kept its work is a small fraction of the walk's.

Customer. With 1% kept, every plan takes 3.1 s to 3.6 s and Memento 10.1 s. A customer that is not kept has no witness, so its walk visits its orders and their lineitems before it fails: 29.9 reads and 7.6 node arrivals per customer. With 30% kept, more walks go deep before they fail, the reads rise to 56.8 per customer, and Memento trails Bespoke$_{Duck}$ by 2.6×. With 60% kept, a kept customer stops at its first witness. Its reads drop to 36.9, and Memento takes 11.3 s, 1.2× behind Bespoke$_{Duck}$ and 4.2× ahead of the fastest PostgreSQL method. From 1% to 60% kept, Memento thus slows by 1.1×, and the plans by 3.0× to 17×. A policy that hides most rows thus favours a plan, and the gap closes as the policy shows more rows.

D.2.3 Depth. Figure 16 keeps 60% of the rows and adds one step of the chain at a time.

Lineitem. PG-RLS grows from 221 s at depth 1 to 830 s at depth 7, whereas Memento stays between 21.8 s and 31.8 s. Seven steps cost Memento 1.5× the time of one, against 1.7× for Verbatim$_{Duck}$, 4.3× for Bespoke$_{Duck}$, and 24× for Verbatim$_{PG}$. Through depth 5 the walk reads 4.0 rows per lineitem whatever the depth. Each new relation is entered by a key, so its decision is kept at the rank of that key (Section 5.2) and every later lineitem with the same key reads it instead of walking. The plans jump at step 6, which joins the orders and compares a price with a total: Verbatim$_{PG}$ goes from 38.8 s to 576 s and Bespoke$_{Duck}$ from 4.6 s to 13.2 s. Memento adds 4.3 s at this step. A lineitem shares its order with few others, so the order step is reused less than the steps before it, and node arrivals rise from 1.21 to 1.83 per lineitem. Memento is faster than the three PostgreSQL methods at every depth. Bespoke$_{Duck}$ stays ahead, by

6.3× at depth 1 and 2.1× at depth 7, since the walk visits all 60M lineitems, whereas $\text{Bespoke}_{\text{Duck}}$ hashes only the rows that pass the filters.

Customer. The first two steps follow a customer's orders and their lineitems. Here Memento is the fastest method, 1.7× ahead of $\text{Bespoke}_{\text{Duck}}$ at depth 1 and 1.5× at depth 2, since a customer stops at its first qualifying order or lineitem. Step 3 enters partsupp through each lineitem's part and supplier, a pair that rarely repeats within a customer, so the reads per customer rise from 4.8 to 23.2 and Memento falls 1.4× behind $\text{Bespoke}_{\text{Duck}}$. Steps 4 and 5 add part and supplier, and Memento grows to 10.9 s. It then stays flat, because nation and region have 25 and 5 rows, whose decisions are read rather than walked, and the reads grow only from 35.0 to 36.9 per customer. The PostgreSQL methods reach 44 s to 57 s from depth 4 on.

D.2.4 Parametricity and Width. Table 18 gives the times of Figures 9 and 10, with the memory of the two methods that finish.

Table 18: Nested policies at SF 10. Time in seconds, peak memory in GiB. The PostgreSQL methods and $\text{Verbatim}_{\text{Duck}}$ finish only as Figures 9 and 10 show.

	$\text{Bespoke}_{\text{Duck}}$		Memento		
policy	time	peak_{mem}	time	peak_{mem}	reads/line
P1	440	64	387	24	7.0
P2	440	66	316	30	6.4
P3	422	65	319	30	6.9
P4	442	65	364	31	7.2
P5	460	65	405	31	7.2
P6	427	64	319	31	6.5
P7	469	64	439	31	6.9
P8	444	64	416	32	8.4
W2	119	38	135	22	3.6
W4	165	45	223	23	6.6
W6	270	52	247	26	6.4
W8	439	65	304	28	6.7
W10	1051	78	553	31	7.7

What finishes. Only $\text{Bespoke}_{\text{Duck}}$ and Memento finish every nested policy at SF 10 (Table 18). $\text{Verbatim}_{\text{PG}}$ and PG-RLS finish W2, in about 1,740 s, and time out on every other policy. $\text{Bespoke}_{\text{PG}}$ times out on every policy. $\text{Verbatim}_{\text{Duck}}$ finishes W2 and W4 and runs out of memory on every other policy.

Parametricity. Across the eight bindings, Memento takes 316 to 439 s and $\text{Bespoke}_{\text{Duck}}$ 422 to 469 s, so Memento is faster on all eight. P1 runs first after the caches are cleared, and the other seven follow warm. Every binding starts with an empty decision store, so a warm run gains only from cached pages, as it does for DuckDB. The walk reads 6.4 to 8.4 rows per lineitem, although the bindings differ in up to 22 predicates (Table 5): a binding changes how far each walk goes, not where it enters. $\text{Bespoke}_{\text{Duck}}$ peaks at 64 to 66 GiB, and Memento at 24 to 32 GiB.

Width. From W2 to W10, the policy grows from 2 to 10 branches. $\text{Bespoke}_{\text{Duck}}$ slows by 8.9× and its peak memory grows from 38 to 78 GiB, as every branch adds joins to its plan. Memento slows by 4.1× and stays between 22 and 31 GiB, since its reads per lineitem only grow from 3.6 to 7.7. It is slower than $\text{Bespoke}_{\text{Duck}}$ on W2 and W4, where the plan has few joins, and faster from W6 on, 1.9× at W10.

D.3 Scalability

Section 6.4 runs the TPC-H policies at four scales. Table 19 gives the tier medians of Figure 11 at each scale.

Table 19: Memento on TPC-H at four scales, warm.

	median time (s)				ns per lineitem			
tier	SF 0.1	SF 1	SF 10	SF 100	SF 0.1	SF 1	SF 10	SF 100
simple	0.01	0.115	1.35	13.5	17	19	23	22
intermediate	0.02	0.209	3.06	35.2	34	35	51	59
complex	0.021	0.232	3.28	43.9	36	39	55	73

Table 20: Work of Memento per lineitem on TPC-H, median per group.

		ns per lineitem				witness reads per lineitem			
policies		SF 0.1	SF 1	SF 10	SF 100	SF 0.1	SF 1	SF 10	SF 100
filter only	3	11	10	12	12	0.00	0.00	0.00	0.00
with branches	27	30	33	48	58	0.26	0.27	0.27	0.27

Growth with scale. From SF 0.1 to SF 10, the lineitems grow 100×, and the medians of the simple, intermediate, and complex tiers grow 132×, 151×, and 153×, each tenfold step costing between 10.3× and 14.6×. From SF 10 to SF 100, the medians grow 10.0×, 11.5×, and 13.4×, to 13.5, 35.2, and 43.9 s. Over the whole range from SF 0.1 to SF 100, 1,000× more lineitems thus cost 1,321×, 1,731×, and 2,046× more time. Per lineitem, the simple tier grows from 17 to 23 ns at SF 10 and stays at 22 ns at SF 100. The intermediate and complex tiers grow from 34 and 36 ns at SF 0.1 to 51 and 55 ns at SF 10, and to 59 and 73 ns at SF 100.

Work per lineitem. The walk does the same work per lineitem at every scale (Table 20). In 23 of the 30 policies, its witness reads per lineitem change by less than 5% from SF 0.1 to SF 100, and 16 policies store the same number of decisions at every scale, since their decisions are keyed by nations, regions, or other attributes whose domain does not grow. Time per lineitem changes little from SF 0.1 to SF 1, and grows from SF 1 to SF 10 by a median of 1.43× for the policies with branches, but only 1.13× for the three that only filter the target. From SF 10 to SF 100, it grows by a further 1.19× for the policies with branches, and 1.03× for the three. The added time is thus in the lookups, not in the scan. This is what we expect from the cache: the machine has 60 MB of last-level cache, and a rank column of `orders` takes 6 MB at SF 1, 60 MB at SF 10, and 600 MB at SF 100, so lookups into it miss more and more often.

D.4 Space Overhead

Section 6.5 gives the size of the artifacts. The decision store adds to them only while a query runs.

The store. Table 24 shows what the walk keeps. A policy needs few decisions, a median of a few thousand for 60M lineitems in the complex tier, and reads less than one witness per target row on MIMIC and TPC-H. On IMDB, it reads about one, since most of its decisions belong to a single row (Section D.1). Such a decision is never asked again, so the walk does not store it: 8 of the 10 intermediate JOB policies store no decision at all, which is why their median is 0. The size of the store follows the ranks the walk can enter, not the decisions it writes, since the store keeps one slot for every rank. T11, for example, writes 25 decisions and keeps 81 MB.

Table 21: TPC-H policies, warm time in seconds, fastest in bold. f: filter on the target. Shared join: the relation two branches join first, ∃/¬∃ if one of them is negated. Many-to-many: a join of two non-key attributes. TO: timeout. OOM: out of memory.

tier	policy	rel.	branches	shared join	many-to-many	kept, %	PG-RLS	DBMS-O	Verbatim$_{PG}$	Bespoke$_{PG}$	Verbatim$_{Duck}$	Bespoke$_{Duck}$	MEMENTO
simple	T01	1	f			29	4.23	1.02	4.23	4.23	1.39	5.72	**0.621**
	T02	1	f			15	4.12	6.12	4.22	4.23	**1.02**	1.91	1.30
	T03	1	f			2.2	4.38	3.68	3.28	3.29	**0.185**	0.188	0.698
	T04	2	∃			20	10.6	6.69	7.90	6.98	5.04	2.43	**1.44**
	T05	2	¬∃			70	20.3	9.79	20.7	33.2	6.25	**2.79**	28.8
	T06	2	f∧¬∃			2.2	10.9	8.71	9.87	19.0	4.03	4.23	**1.23**
	T07	2	f∨∃			13	TO	TO	TO	12.0	18.7	0.937	**0.714**
	T08	2	f∧∃		yes	3.4	3.43	17.6	4.32	4.82	**1.10**	3.50	1.40
	T09	2	f∨¬∃		yes	82	10.3	TO	10.4	62.5	4.06	3.58	**2.06**
	T10	2	∃			0.1	7.08	0.129	0.048	**0.048**	2.36	0.451	2.21
intermediate	T11	4	∃			4.0	8.03	5.52	3.16	3.19	3.24	**0.581**	2.65
	T12	4	∃∧¬∃	orders (∃/¬∃)		16	17.6	10.7	12.5	18.7	6.25	3.64	**2.87**
	T13	5	∃∧∃	orders	yes	0.8	64.6	17.4	63.3	109	TO	17.0	**3.10**
	T14	3	∃			13	TO	1.01	TO	14.2	68.2	**0.483**	0.558
	T15	4	∃∨∃		yes	27	10.6	TO	10.8	78.9	3.35	2.06	**1.90**
	T16	4	¬∃∧¬∃		yes	69	14.5	11.8	70.7	27.9	4.55	**2.20**	3.96
	T17	4	∃∨¬∃	orders (∃/¬∃)		84	19.4	60.0	19.3	22.0	10.2	5.10	**3.50**
	T18	4	¬∃∨¬∃	orders		96	15.0	6.04	15.3	20.7	9.95	4.73	**3.32**
	T19	5	∃∧¬∃	orders (∃/¬∃)		10	19.7	9.58	14.4	22.6	5.78	3.53	**3.03**
	T20	4	∃∨¬∃			92	TO	TO	TO	20.1	22.8	**1.94**	3.20
complex	T21	6	∃		yes	0.8	299	13.4	297	56.7	12.2	12.2	**3.30**
	T22	8	∃∧¬∃	orders (∃/¬∃)		0.1	273	86.7	244	7.50	9.97	**0.751**	2.07
	T23	7	∃∧¬∃∧¬∃	orders (∃/¬∃)	yes	3.6	493	277	288	116	TO	17.1	**7.90**
	T24	7	¬∃∨¬∃	orders		90	278	307	277	158	20.0	18.1	**5.53**
	T25	7	∃∧¬∃	supplier (∃/¬∃)		1.2	125	13.9	144	10.6	10.9	3.59	**2.30**
	T26	8	∃∨∃			36	18.4	269	18.5	90.7	6.82	3.45	**3.26**
	T27	8	¬∃∧¬∃	orders	yes	91	298	1004	319	77.8	15.9	16.9	**6.56**
	T28	8	(∃∨∃)∧¬∃	orders (∃/¬∃)	yes	15	TO	TO	TO	134	OOM	18.1	**4.35**
	T29	6	∃∨¬∃		yes	81	17.5	TO	17.6	90.5	6.30	3.47	**2.84**
	T30	8	∃∧∃	orders	yes	<0.1	62.9	93.6	61.5	352	11.9	13.9	**2.34**

Table 22: MIMIC policies, warm time in seconds, fastest in bold. f: filter on the target. Shared join: the relation two branches join first, ∃/¬∃ if one of them is negated. Many-to-many: a join of two non-key attributes.

tier	policy	rel.	branches	shared join	many-to-many	kept, %	PG-RLS	DBMS-O	Verbatim$_{PG}$	Bespoke$_{PG}$	Verbatim$_{Duck}$	Bespoke$_{Duck}$	MEMENTO
simple	M01	1	f			2.2	0.003	0.057	0.004	0.004	0.003	0.003	**0.002**
	M02	1	f			<0.1	**0.006**	0.067	0.007	0.007	0.011	0.01	0.008
	M03	1	f			5.6	0.025	0.116	0.025	0.025	0.014	0.014	**0.01**
	M04	2	f∨∃			16	0.298	0.296	0.301	0.105	0.016	0.011	**0.008**
	M05	2	∃			0.2	0.853	0.624	0.13	0.119	0.034	0.015	**0.005**
	M06	2	f∨¬∃			76	0.293	0.157	0.289	0.118	0.019	0.013	**0.009**
	M07	2	∃		yes	23	0.261	0.146	0.062	0.061	0.016	**0.011**	0.02
	M08	2	f∧∃		yes	1.7	0.121	0.127	0.023	0.124	0.013	0.023	**0.006**
	M09	2	f∧¬∃		yes	7.7	0.047	0.148	0.056	0.059	0.018	0.016	**0.011**
	M10	2	¬∃		yes	83	14.0	0.327	0.597	0.269	**0.02**	0.023	0.037
intermediate	M11	4	∃∧∃	icustays		1.7	0.379	0.076	**0.005**	0.005	0.011	0.007	0.006
	M12	3	∃			3.4	0.326	0.141	0.086	0.042	0.017	0.007	**0.007**
	M13	4	¬∃∨¬∃	admissions		96	0.364	0.128	0.369	0.103	0.024	0.016	**0.009**
	M14	3	∃		yes	88	2.92	0.167	0.139	0.254	**0.023**	0.025	0.193
	M15	3	∃∨∃		yes	79	0.335	0.106	0.328	0.11	0.025	0.019	**0.004**
	M16	4	∃∧¬∃	diagnoses_icd (∃/¬∃)	yes	11	0.342	0.104	0.027	0.029	0.016	0.012	**0.003**
	M17	4	∃∧¬∃	procedureevents (∃/¬∃)	yes	39	2.74	0.161	0.259	0.203	**0.025**	0.026	0.033
	M18	4	∃∧¬∃	inputevents (∃/¬∃)	yes	14	0.428	0.171	0.152	0.159	0.025	0.019	**0.013**
	M19	4	¬∃∧¬∃		yes	74	0.447	0.174	0.18	0.168	0.055	**0.015**	0.106
	M20	3	¬∃∨∃		yes	93	0.419	0.824	0.421	0.324	0.042	0.027	**0.016**
complex	M21	7	∃∧∃			2.8	0.448	0.151	0.032	0.043	0.017	0.011	**0.005**
	M22	9	∃∧¬∃	admissions (∃/¬∃)	yes	5.0	1.35	0.297	0.582	0.096	0.017	0.01	**0.004**
	M23	10	∃		yes	15	6.95	0.271	0.162	0.132	0.035	0.152	**0.009**
	M24	9	∃∨∃		yes	17	5.55	0.528	5.63	0.227	**0.029**	0.062	0.14
	M25	8	∃∨¬∃	admissions (∃/¬∃)	yes	52	2.29	0.403	2.30	0.233	0.038	**0.034**	0.088
	M26	9	(∃∨∃)∧¬∃	admissions (∃/¬∃)	yes	61	2.26	0.31	2.56	0.291	0.045	0.041	**0.025**
	M27	7	¬∃∧¬∃	admissions		32	0.42	0.154	0.066	0.096	0.021	0.016	**0.004**
	M28	9	∃∧¬∃	icustays (∃/¬∃)	yes	3.6	7.10	0.26	0.186	0.222	**0.021**	0.031	0.264
	M29	6	¬∃∨¬∃		yes	90	16.7	2.48	16.7	0.21	0.035	0.027	**0.007**
	M30	8	∃∨(∃∧¬∃)∨∃		yes	77	3.40	0.502	3.42	0.425	0.072	**0.052**	0.226

D.5 Per-Policy Results

Tables 21, 22, and 23 record each of the 90 policies: its shape, the share of target rows it keeps, and the time of every method. MEMENTO finishes every policy and is the fastest method on 20 of the 30 TPC-H policies, 19 of the 30 MIMIC policies, and 9 of the 30 JOB policies, where Bespoke$_{Duck}$ is fastest on 18 (Section D.1 explains why). Of the baselines, only Bespoke$_{PG}$ and Bespoke$_{Duck}$ finish every policy, PG-RLS fails on 6, DBMS-O fails on 6, Verbatim$_{PG}$ fails on 4, and Verbatim$_{Duck}$ on 3.

The JOB families show how time grows with the size of a policy. In 20a, 23c, and 26c, the full query adds relations to the cut version with two. Bespoke$_{Duck}$ then slows by 16 to 133×, and MEMENTO

Table 23: IMDB policies, warm time in seconds, fastest in bold. No policy shares a join between branches or joins two non-key attributes. TO: timeout.

tier	policy	rel.	branches	kept, %	PG-RLS	DBMS-O	Verbatim$_{PG}$	Bespoke$_{PG}$	Verbatim$_{Duck}$	Bespoke$_{Duck}$	MEMENTO
simple	12c$_2$	2	∃	4.0	4.26	114	0.706	0.629	0.588	**0.318**	0.547
	13b$_2$	2	∃	26	0.361	0.096	0.045	0.045	0.038	**0.007**	0.022
	14c$_2$	2	∃	35	1.09	0.468	0.304	0.304	0.079	0.043	**0.038**
	20a$_2$	2	∃	19	1.69	0.484	0.107	0.109	0.063	**0.016**	0.033
	21c$_2$	2	∃	26	5.78	152	1.25	1.34	0.802	**0.532**	0.905
	23c$_2$	2	∃	21	1.49	0.472	0.159	0.156	0.063	**0.028**	0.035
	25c$_2$	2	∃	5.7	0.618	3.60	0.473	**0.345**	0.606	0.383	1.43
	26c$_2$	2	∃	11	1.26	0.465	0.103	0.102	0.059	**0.017**	0.035
	3a$_2$	2	∃	1.1	4.10	4.77	0.513	0.487	0.626	**0.34**	0.641
	3c$_2$	2	∃	17	6.62	141	0.943	1.03	0.771	**0.49**	1.17
intermediate	15c$_5$	5	∃	11	17.9	3.11	6.33	2.31	0.767	**0.374**	0.63
	1a	5	∃	<0.1	13.3	0.5	**0.009**	0.185	0.191	0.018	0.406
	21c$_5$	5	∃	15	9.74	155	1.71	1.69	0.775	**0.534**	1.03
	25c$_5$	5	∃	3.0	TO	3.91	0.521	0.396	0.327	**0.102**	0.762
	2d	5	∃	1.0	TO	0.596	0.394	0.569	0.259	**0.077**	1.05
	3c	4	∃	6.6	6.51	144	5.45	1.90	0.888	**0.583**	1.26
	4c	5	∃	5.6	10.4	2.29	2.26	1.78	0.675	**0.272**	0.296
	5c	5	∃	9.1	8.98	143	1.32	1.28	0.761	**0.539**	1.01
	6d	5	∃	0.5	5.74	2.46	4.49	8.22	0.945	1.42	**0.271**
	7c$_5$	5	∃	0.3	5.01	3.35	1.89	4.66	0.749	0.504	**0.19**
complex	13b	9	∃	2.3	21.2	3.96	9.14	1.61	0.855	**0.253**	0.606
	19c	10	∃	4.2	55.4	40.8	91.4	9.29	12.2	**1.36**	2.51
	20a	10	∃	0.2	11.3	2.65	2.13	12.2	0.758	2.11	**0.137**
	22c	11	∃	<0.1	5.20	5.01	1.94	1.28	0.806	**0.227**	0.392
	23c	11	∃	0.2	10.6	18.0	121	1.94	9.11	0.449	**0.093**
	24a	12	∃	<0.1	285	4.44	60.9	9.00	**0.666**	1.35	1.94
	26c	12	∃	<0.1	7.73	1.40	1.90	12.4	0.633	2.15	**0.081**
	28c	14	∃	<0.1	4.02	5.36	0.68	1.20	0.641	0.249	**0.17**
	30c	12	∃	0.2	18.4	18.5	5.71	5.97	0.79	1.47	**0.172**
	31c	11	∃	0.8	41.3	171	168	13.9	9.45	2.83	**1.17**

Table 24: The decision store, median per tier. Reads per row: witness tuples the walk reads per target row.

dataset	tier	witness reads	decisions	store	reads per row
MIMIC	simple	742	2	630B	<0.01
	intermediate	4.0K	316	4.1KB	0.01
	complex	22K	902	35KB	0.03
TPC-H	simple	4.9M	6	954B	0.08
	intermediate	18M	1.3K	119MB	0.30
	complex	20M	2.5K	155MB	0.33
IMDB	simple	2.4M	3	487B	0.93
	intermediate	4.0M	0	624B	1.59
	complex	1.8M	32K	35KB	0.70

only by 2.3 to 4.1×, so that MEMENTO is the fastest method on all three full queries.

E Implementation

MEMENTO consists of two programs, whose source and build scripts are in the repository: an offline builder that creates the rank artifacts, and an extension of PostgreSQL 16, written in C and C++, that enforces the policies at query time. Figure 17 shows how they meet. This appendix describes how a query runs, how the artifacts are laid out, how a policy is compiled, and how the decision store is kept, and ends with the limitations of the current implementation.

E.1 Query Execution

The extension registers a planner hook and a hook on the access paths of each relation. When the planner meets a scan of a relation that a policy protects, the target of the policy, it replaces the scan by a custom scan that wraps the original one. The custom scan keeps the cost of the original scan, so the plan of the query does

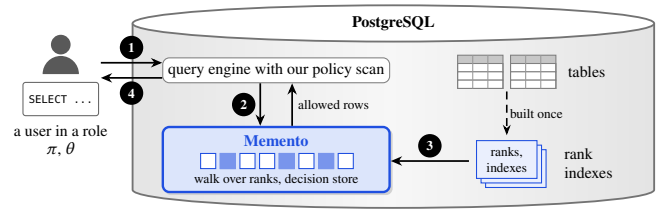


Figure 17: Memento inside PostgreSQL. The policy scan of a query (1) asks Memento (2), which walks the rank indexes (3), and only allowed rows reach the query (4).

not change. A scan of a protected relation that cannot be wrapped is refused, so a query never reads the target without its policy.

When the custom scan starts, the extension loads the artifacts of the policy from the table `memento.files`, compiles the template and the binding (Section E.3), and decides all rows of the target in one evaluation. The result is a bitmap with one bit per row id of the target. The extension turns it into words addressed by heap block and line offset, and from then on the scan returns a tuple of the original scan only if its bit is set. Loading the artifacts is not part of the reported times, which start with the compilation of the policy.

Figure 18 shows how the time of the 30 TPC-H policies at SF 10 divides among these phases. Compiling takes a median of 0.6 ms and at most 24 ms. The walk takes a median of 78% of the time on simple policies and 94% on complex ones. Within the walk, evaluation reads the data for decisions not yet stored, and reuse answers target rows from stored decisions. Reuse takes a median of a quarter of the walk, although it answers most target rows: in T10, a kept verdict decides 97% of the lineitems in 27% of the walk, and the first arrivals at 2M parts take the rest. Reuse dominates where few decisions answer many rows, with 93% of the walk in T05 and 87% in T25, and in T07 one boundary decides 54M of the

60M lineitems. Turning bits into row ids takes longer the more rows are allowed, 69 ms for the 0.5M lineitems that T13 allows and 282 ms for the 17M of T01.

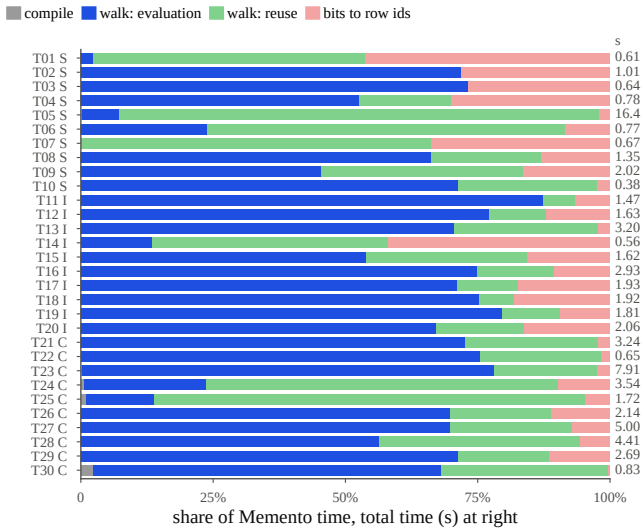


Figure 18: Time of Memento per phase on TPC-H at SF 10, with the total in seconds. S, I, C: simple, intermediate, complex.

E.2 Artifacts

The builder reads from the templates which columns they mention and which pairs of columns they compare, never an operator or a constant, so a new binding needs no new artifact (R4). Columns that a template compares with each other are merged into one rank universe, ranked over the union of their values. For each universe, the builder writes a dictionary, the sorted distinct values. For each column, it writes a forward map, the rank of every row in heap order, and an inverted index, which lists the row ids of the column grouped by rank and stores one offset per rank of the universe into this list (Section 5.1). The files are stored in `memento.files`, one bundle per database. At compile time, a constant of the binding is placed by one binary search in the dictionary of its universe, which gives the half-open range of ranks equal to it, and each operator becomes a test on ranks.

Merging enlarges a universe beyond the values of each column (Figure 19). The six money columns of TPC-H form one universe of 13.3M ranks, while the retail prices of parts take only 31,681 distinct values, 421× fewer. Since an inverted index stores one offset per rank of its universe, merging adds 265 MB to a bundle of 9.1 GB at SF 10, 2.9%. Any state indexed by rank is sized by the universe as well, which is why the decision store bounds its dense arrays by a memory budget (Section E.4).

E.3 Policy Compilation

The compiler turns the template into the decision graph of Section 4.3, with one node per table occurrence and the root at the target. For every node it records the connection through which the walk enters it, the slices it iterates, its Rest, and the filled operands that the Rest reads from outside, which together form the layout

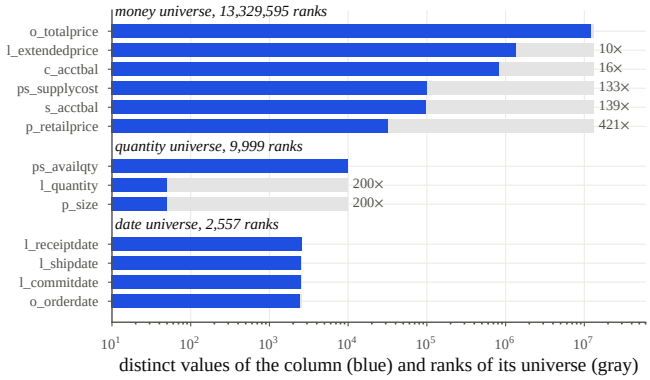


Figure 19: TPC-H universes with more than one column at SF 10, and how many ranks each column leaves unused.

of the node's key. The binding then sets the operator of each atom and places its constants among the ranks, as above.

A policy runs on one of two execution paths. A policy whose quantifiers sit directly under the Boolean form of the root runs on the compiled target loop of Appendix C, in which every branch of the root reads a cell kept for its entering rank and falls back to the walk when the cell is empty. A policy with quantifiers nested inside other quantifiers runs on the nested walk, which walks the scopes of the graph from each block of target rows and shares the nodes that several branches enter (Section 5.3). Both paths use the same artifacts, the same keys, and the same decision store. The compiler also decides which of the extra optimizations of Appendix C apply, each from a property of the policy and its artifacts.

E.4 Decision Store

Each node that stores decisions has its own store, keyed by the rank through which the walk enters the node and by the filled operands of its Rest (Section 4.3). When a key consists of one rank, the store is a dense array indexed by that rank, with two bits per decision, if it fits in the memory budget, 32 GiB in our experiments. Otherwise, the store is a hash table that holds only the keys the walk reaches.

A store pays off only when its keys repeat, and a repeated entering rank does not imply a repeated key, since the filled operands differ from one target row to the next. Figure 20 shows this on the 87 stores of the 13 nested policies at SF 10. The stores at the part key lie on the diagonal: their keys repeat as often as their entering ranks, up to 27.5 times, and they answer up to 96% of their probes. The stores at the order key, whose Rest reads a value of the lineitem, lie below it, with keys that repeat at most 2.3 times and answer at most 57% of the probes. The stores at the customer, partsupp, and supplier keys never hit, although their entering ranks repeat up to 282 times.

Since the artifacts alone cannot tell which stores will pay, every store counts its probes and its hits. A miss costs a probe and a record on top of the walk it does not save, so a store that does not hit switches itself off for the rest of the query, and the walk evaluates its node without probing from then on. In every nested policy, the stores that never hit switch off after at most 131,072 probes, out of up to 60M arrivals.

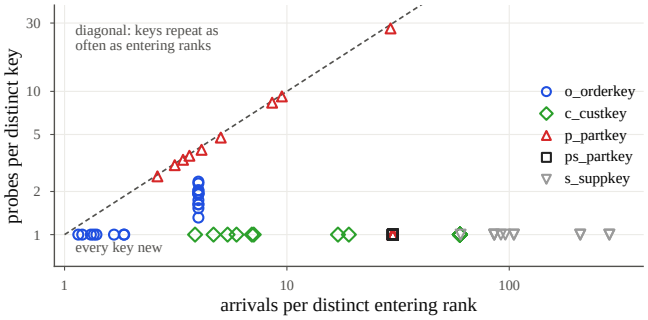


Figure 20: How often the keys of each decision store repeat, against how often its entering rank repeats. Nested policies at SF 10.

E.5 Limitations

Three restrictions keep the implementation simple.

No state across queries. The decision store lives for one query. Each binding of the parametricity experiments starts with an empty store, although 16 of their 64 branches repeat exactly across bindings. Keeping decisions across queries would require invalidating them on every update of the data, which a store per query avoids.

One core per query. MEMENTO decides the target in blocks of 1,024 rows, one block after another. The blocks could run in parallel, but they share one store, so a parallel evaluation would have to share or partition it.

Artifacts rebuilt after updates. The artifacts are built offline and rebuilt when the data changes. Ranks are dense, so inserting a new value shifts every larger rank of its universe, and an incremental build would need gaps between ranks or a second level of ranks for new values.